



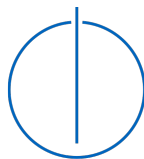
DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Generalized Monocular Depth Prediction for Autonomous Driving

Melih Görgülü





DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Generalized Monocular Depth Prediction for Autonomous Driving

Generalisierte monokulare Tiefenvorhersage für Autonomes Fahren

Author:	Melih Görgülü
Supervisor:	Prof. Dr. Daniel Cremers
Advisor:	Johannes Michael Meier
Submission Date:	17/06/2024



I confirm that this master's thesis in informatics is my own work and I have documented all sources and material used.

Munich, 17/06/2024

Melih Görgülü

Abstract

Rapid improvements in autonomous driving technology have increased the demand for reliable and accurate perception systems. Monocular 3D object detection, which identifies and detects objects using a single camera, offers a cost-effective and scalable solution. Despite its promise, the lack of depth information presents a significant challenge, even with abundant monocular autonomous driving data. Consequently, predicting accurate depths for objects is crucial for effective monocular 3D object detection. However, variability in camera setups and views makes this task difficult. This thesis addresses these challenges by developing a generalized monocular depth prediction method, improving depth estimation accuracy and robustness across different views.

The research explores three methods: virtual depth, full rotation learning, and keypoint exploitation. The virtual depth method mitigates depth-scale ambiguity by incorporating focal length into the optimization process, which provides a unified target for depth estimation and enhances detection accuracy across various camera configurations. The full rotation learning method enables perspective-invariant learning by incorporating a 3×3 rotation matrix in allocentric form. In addition, the full rotation matrix allows the achievement of depth estimations from keypoints using the Perspective-n-Point method, which can provide depth clues for final depth prediction. Keypoint exploitation investigates the use of spatial information and depth clues from keypoints to enhance depth prediction.

The methods were evaluated on the KITTI and ROPE3D datasets, demonstrating improved detection accuracy and robustness. While virtual depth and full rotation learning showed consistent benefits, keypoint exploitation shows potential but requires further refinement. Combining different depth estimations using an uncertainty-based approach proved that it is effective, although it still suggests room for future optimization.

This thesis contributes to the field of autonomous driving by enhancing monocular 3D object detection capabilities, offering more accurate and reliable depth predictions across diverse scenarios, and providing a direction for further research to develop safer autonomous vehicles.

Kurzfassung

Die rasanten Verbesserungen in der Technologie des autonomen Fahrens haben die Nachfrage nach zuverlässigen und genauen Wahrnehmungssystemen erhöht. Die monokulare 3D-Objekterkennung, bei der Objekte mit einer einzigen Kamera identifiziert und erkannt werden, bietet eine kostengünstige und skalierbare Lösung. Trotz dieser vielversprechenden Lösung stellt das Fehlen von Tiefeninformationen eine große Herausforderung dar, selbst bei einer Vielzahl von monokularen Daten für autonomes Fahren. Daher ist die Vorhersage genauer Objektiefen für eine effektive monokulare 3D-Objekterkennung von entscheidender Bedeutung. Die Variabilität der Kameraeinstellungen und -ansichten macht diese Aufgabe jedoch schwierig. In dieser Arbeit werden diese Herausforderungen durch die Entwicklung einer verallgemeinerten monokularen Tiefenvorhersagemethode angegangen, die die Genauigkeit und Robustheit der Tiefenschätzung über verschiedene Ansichten hinweg verbessert. In der Forschung werden drei Methoden untersucht: virtuelle Tiefe, Lernen durch vollständige Rotation und Ausnutzung von Schlüsselpunkten. Die Methode der virtuellen Tiefe entschärft die Mehrdeutigkeit der Tiefenskala, indem sie die Brennweite in den Optimierungsprozess einbezieht, was ein einheitliches Ziel für die Tiefenschätzung bietet und die Erkennungsgenauigkeit bei verschiedenen Kamerakonfigurationen verbessert. Die Methode des vollständigen Rotationslernens ermöglicht perspektiveninvariantes Lernen durch die Einbeziehung einer 3×3 -Rotationsmatrix in allozentrischer Form. Darüber hinaus ermöglicht die vollständige Rotationsmatrix die Erzielung von Tiefenschätzungen aus Keypoints unter Verwendung der Perspective-n-Point-Methode, die Tiefenhinweise für die endgültige Tiefenvorhersage liefern kann. Die Nutzung von Keypoints untersucht die Verwendung von räumlichen Informationen und Tiefenhinweisen von Keypoints zur Verbesserung der Tiefenvorhersage. Die Methoden wurden anhand der KITTI- und ROPE3D-Datensätze bewertet und zeigten eine verbesserte Erkennungsgenauigkeit und Robustheit. Während das Erlernen der virtuellen Tiefe und der vollständigen Rotation durchgängig Vorteile brachte, zeigt die Ausnutzung von Schlüsselpunkten Potenzial, muss aber weiter verfeinert werden. Die Kombination verschiedener Tiefenschätzungen unter Verwendung eines auf Unsicherheit basierenden Ansatzes erwies sich als effektiv, obwohl sie noch Raum für künftige Optimierungen bietet. Diese Arbeit leistet einen Beitrag zum Bereich des autonomen Fahrens, indem sie die monokularen 3D-Objekterkennungsfähigkeiten verbessert, genauere und zuverlässigere Tiefenvorhersagen in verschiedenen Szenarien bietet und eine Richtung für weitere Forschung zur Entwicklung sicherer autonomer Fahrzeuge vorgibt.

Contents

Abstract	iii
Kurzfassung	iv
1 Introduction	1
2 Related Works	3
3 Background	6
3.1 Monocular 3D Object Detection	6
3.2 Base Detector	7
3.2.1 Feature Backbone	8
3.2.2 3D Bounding Box Regression	8
3.2.3 Decoding the Predicted 3D Bounding Box	9
3.2.4 The Auxiliary Context Regression	9
4 Methods	10
4.1 Problem Analysis	10
4.2 Virtual Depth	10
4.3 Predicting Full Rotation	12
4.4 Exploiting the Information in Keypoints for Depth Estimation	13
4.4.1 Adding Auxiliary Keypoints	13
4.4.2 Keypoint-to-Center Offset Learning	14
4.4.3 Dense 3D Bounding Box Property Prediction Using Keypoints	14
4.4.4 Associating Keypoint Predictions with Objects	16
4.4.5 Depth Estimation from Keypoints Using Perspective-n-Point	18
4.4.6 Learning Uncertainty for Keypoint-Based Perspective-n-Point Depth Predictions	20
4.4.7 Combining Dense Depth Estimations	21
5 Analysis	23
5.1 Learning Keypoint Heatmaps for 3D Features	23
5.2 Keypoint Depth Heatmap Prediction Analysis	24
5.3 Learning Uncertainties for Formula Depth Predictions	27
5.4 Combining Dense Estimations	29

6	Experiments	31
6.1	Datasets	31
6.1.1	KITTI 3D	31
6.1.2	ROPE3D	31
6.2	Evaluation Metrics	31
6.3	Implementation Details	32
6.4	Qualitative Results	33
6.4.1	Adding Auxiliary Keypoints	33
6.4.2	Keypoint2Center Offset Learning	33
6.4.3	Predicting 2D Keypoint Locations	34
6.5	Ablation Studies	34
6.5.1	Virtual Depth and Full Rotation	35
6.5.2	Dense Depth Estimation	36
7	Discussion	38
8	Conclusion	41
	List of Figures	43
	List of Tables	44
	Acronyms	45
	Bibliography	46

1 Introduction

The rapid advancements in autonomous driving technology necessitate the development of reliable and accurate perception systems. Monocular 3D object detection, which identifies and localizes the object in the scenes using only a single camera, is a critical component for autonomous driving due to its cost-effectiveness and scalability. Compared with the Light Detection and Ranging (LiDAR)/stereo-based methods [1, 2, 3, 4], monocular 3D object detection is still a challenging task due to the lack of depth information, making monocular object-level depth estimation naturally ill-posed. The variability in camera installations, focal lengths, and viewing angles further complicate this task, leading to scale-depth ambiguities and inconsistent object detection across different environments.

Despite extensive research in this field, most methods are tailored specifically to certain datasets, creating a need for further research on generalized depth prediction methods that work robustly across different views. Therefore, the primary objective of this thesis is to develop a generalized monocular depth prediction method that effectively addresses the challenges posed by different camera setups and viewing conditions in autonomous driving scenarios. This research aims to improve depth estimation accuracy and robustness in any view, thereby enhancing the overall performance of monocular 3D object detection systems. Key research questions include how to mitigate inconsistent depth predictions caused by varying focal lengths and camera perspectives to enhance object detection accuracy, what methods can achieve perspective-invariant learning for depth estimation, and how object keypoints can improve depth prediction and enhance geometric understanding.

To address these challenges, this thesis proposes three methods for monocular depth estimation. Firstly, to mitigate the impact of depth-scale ambiguity and variable camera intrinsics, the concept of virtual depth [5] is implemented for the current base detector. This approach normalizes the depth estimation process across different camera setups by introducing focal length into the optimization process. Secondly, the full rotation of objects is learned to enhance perspective understanding and utilize the Perspective-n-Point (PnP) method for additional depth estimations from keypoints. This method involves learning a 3x3 allocentric rotation matrix using a 6D continuous parameter instead of learning just the yaw angle, as is commonly done in KITTI. Lastly, the "Exploiting Keypoint Information" approach is motivated by the idea that the spatial information and depth clues from keypoints can enhance the final depth prediction. This method enhances depth prediction by additionally learning object keypoint depth heatmap representation.

The proposed methods are evaluated using the KITTI and ROPE3D datasets, which provide diverse driving scenarios and camera perspectives essential for robust testing. The experimental results and analysis demonstrate the effectiveness of these methods in addressing the identified challenges and improving the overall performance of monocular 3D object

detection systems.

This thesis aims to contribute to the autonomous driving field by advancing monocular 3D object detection capabilities, providing more accurate and reliable depth predictions in different scenarios. This thesis is structured as follows: Chapter 2 reviews the existing literature on monocular 3D object detection and related depth estimation techniques. Chapter 3 provides the necessary background information on monocular 3D object detection and introduces the base detector used in this research. Chapter 4 presents the detailed methodology, including the problem analysis and the proposed approaches for virtual depth, full rotation learning, and keypoint exploitation. Chapter 5 comprehensively analyzes the internal mechanisms and outcomes of our proposed features. Chapter 6 explains the experimental setup and presents the results of various ablation studies conducted to evaluate the effectiveness of the proposed methods. Chapter 7 discusses the thesis with a summary of findings, potential limitations, and suggestions for future research directions, and finally, Chapter 8 summarizes the most vital parts of this thesis and gives a view of future developments.

By addressing the challenges of monocular depth prediction through innovative methods, this research aims to enhance the accuracy and robustness of 3D object detection in autonomous driving, ultimately contributing to the development of safer and more reliable autonomous vehicles.

2 Related Works

Existing 3D Object detection approaches can be broadly categorized into two parts based on their use of either Image or LiDAR data. Although Lidar-based methods generally perform better than image-based methods, the high costs of LiDAR systems make them less feasible in autonomous driving contexts. In contrast, image-based systems provide better scalable solutions since they have easy installation processes and low prices.

In Monocular 3D Object Detection, perceiving 3D environments from 2D images is challenging due to loss of depth information during the camera projection. Therefore, most existing methods utilize extra information to simplify the task, which includes usage of Computer-Aided Design (CAD) models [6, 7, 8], external segmentation [9] and dense depth estimation networks [10, 11, 12, 13]. Mono3D [9] scores and evaluates candidate bounding boxes by exploiting multiple features such as class and instance semantics. This approach utilized external instance and semantic segmentation networks to extract these features effectively. Autoshape [6] learns the meaningful keypoints on the object surface and then uses them as additional geometrical constraints to enhance 3D object detection. The authors predefined several distinguished 3D keypoints on CAD models to supervise the model. Pseudo-LiDAR [12] uses a dense depth estimation network to lift the monocular image into Pseudo-LiDAR representation, then utilizes LiDAR-based detectors. Although the methods that utilize additional information deliver improved performance, they demand more annotations for training and often exhibit reduced generalizability because of dependency on specific data types (CAD models, depth maps, etc.) and different contexts.

Other fully monocular approaches only use a single image for detection and do not use any auxiliary networks or data, which is also the approach we have primarily focused on in this thesis. MonoGRNet [14] is one of the first approaches used for instance-level depth estimation for 3D object detection, which directly predicts the depth of objects with sparse supervision to have enhanced location estimation. M3D-RPN [15] reformulates the 3D detection problem as a 2D-3D anchor generation problem and uses the anchor proposal module to better extract 3D information. They assumed the camera parameters, such as focal length and optical center, remain consistent, which does not hold for the infrastructure side (also called road-side) views. GUPNet [16] estimates the object depth dependent on heights and introduces geometry-guided uncertainty in depth estimation to remedy the error amplification problem caused by projection. To use the geometry guidance, they have preliminary assumptions such as fixed camera position and height, which are only held for car view due to variable camera mounting settings in infrastructure side cameras. MonoFlex [17] proposes a new formulation for object depth estimation by explicitly decoupling the truncated objects. Additionally, it adaptively combines independent estimators using uncertainty-inferred depth. MonoDDE [18] further extends this using the corner keypoints to exploit the depth clues in monocular

images and develops a model which produces 20 depths for each target. MonoCon [19] learns 3D-2D monocular contexts as auxiliary tasks in training. They demonstrated that learning well-posed 2D supervision signals during training helps to improve performance. MonoXiver [20] provides a generic and highly efficient second-stage refinement module for any pre-trained state-of-the-art monocular 3D object detection method. The refinement module generates local 3D anchor space via sampling and performs 3D Box denoising.

The proposed models we mentioned mostly focused on sensors mounted on the vehicle. They assume fixed focal lengths, camera mounting heights, and/or the camera’s optical axis is parallel to the ground. For roadside camera systems, using these methods usually results in poor performance because these systems are installed on a pole with a pitched angle, making the above methods not optimal for roadside scenes. MonoGAE [21] is one of the methods designed specifically for roadside camera systems. It suggests that ground planes have a strong prior knowledge of 3D location due to the fixed installation of cameras in roadside scenarios. The method uses the attention mechanism to generate ground-aware features, which are then fed into the detection head to improve 3D detections. CoBev [22] suggests while depth information encodes geometric cues, the height information better captures the distinct classes of semantic objects. Following this observation, geometry-centric depth and semantic-centric height cues are combined. CBR [23] decoupled the perspective view features into front view and bird eye view using Multi Layer Perceptron (MLP), without using calibration parameters. This approach enables calibration-free 3D detection based on Bird’s-Eye-View (BEV) representation.

Although these methods incorporate some clever approaches, they mostly focus on a single perspective, such as a car or roadside camera view. Methods that adopt a consistent approach across different views are significantly fewer and open to investigation. MonoUni [24] develop obstacle projection models of the vehicle and infrastructure sides, designed such that the vehicle side model is a special case of infrastructure. They propose a method called normalized depth to have a focal length invariant setting. Furthermore, the 3D normalized cube depth optimization target is developed to provide additional depth clues. MonoLSS [25] points out that having high accuracy across different views is challenging because the visual features do not match the learning objectives. To overcome this challenge, they proposed the Learnable Sample Selection (LSS) module, which can adaptively select feature samples.

Monocular 3D object detection has seen significant advancements, largely due to the availability of high-quality datasets that provide the necessary data for training and evaluation. The KITTI [26] dataset is one of the most popular datasets in this field. It provides a comprehensive set of images captured from a car driving around a mid-sized city, including highways, rural areas, and urban environments. The dataset includes various annotations for object detection, tracking, and scene flow, making it a benchmark for evaluating algorithms in this domain.

Another important dataset is NuScenes [27], which offers a rich set of annotations with 3D bounding boxes for object detection. It includes data from different modalities such as cameras, LiDAR, and radar. The scenes selected show a diverse and interesting set of driving maneuvers and traffic situations. Similarly, the Waymo Open Dataset [28] provides

high-resolution images and extensive annotations for 3D object detection. Captured from autonomous vehicles, it includes diverse driving scenarios from various locations.

ApolloScape [29] provides a comprehensive dataset focusing on complex urban environments with high-quality images and detailed 3D annotations, making it valuable for testing monocular 3D detection algorithms. Similarly, the BDD100K dataset [30] offers an extensive collection of images and annotations across diverse driving conditions, weather, times of day, and locations, aiding in training robust models for various real-world scenarios.

On the other hand, The ROPE3D dataset [31] is a highly valuable resource for infrastructure-side applications. The dataset consists of 50k images and over 1.5M 3D objects in various scenes, which are captured under different settings, including various cameras with ambiguous mounting positions, camera specifications, and viewpoints. These diverse ranges of object poses and perspectives are crucial for training models to recognize objects from various angles and distances. The dataset includes urban, suburban, and rural environments, making it a comprehensive resource to improve detection accuracy and robustness in real-world scenarios. Another notable dataset is DAIR-V2X-I [32], which offers a unique perspective by integrating Vehicle-to-Everything (V2X) communication data with traditional sensor data. This dataset includes extensive annotations and a wide variety of driving scenarios, enhancing the capability to develop and test advanced monocular 3D object detection models that operate efficiently in connected and autonomous vehicle environments. Integrating V2X data improves situational awareness and decision-making processes in autonomous driving systems.

In this thesis, we have used the KITTI dataset for car view and the ROPE3D dataset for infrastructure side view to leverage their specific advantages for our research. The KITTI dataset provides a well-established benchmark with diverse driving scenarios, essential for evaluating and training our models in typical car-mounted camera settings. Conversely, the ROPE3D dataset is tailored for infrastructure applications, offering detailed annotations and diverse perspectives for addressing roadside camera systems' unique challenges. Utilizing both datasets allows us to develop robust monocular 3D object detection models that perform effectively across different viewpoints, enhancing our approach's overall accuracy and generalizability in real-world scenarios.

3 Background

In this chapter, we provide general background information to help understand the context of this thesis. In Section 3.1, we explain the basics of monocular 3D object detection and offer technical background. In Section 3.2, we explain the base detector we used, its components, and why we chose it for our proposed method.

3.1 Monocular 3D Object Detection

Monocular 3D object detection aims to find every object of interest given a single image. To identify interested objects in a monocular setting, these methods try to estimate the interested objects' 3D bounding box properties and labels (e.g., car, pedestrian, and cyclist). The 3D bounding box can be modeled by 3 properties, i.e., the 3D center location (x, y, z) , dimension (h, w, l) and orientation *yaw angle* (θ) . In the KITTI dataset setting, the roll and pitch angles of objects are set to 0 for the 'car view.' The 'car view' refers to data captured from cameras and sensors mounted on a vehicle, providing a perspective similar to what a driver sees. Setting the roll and pitch angles to 0 means that objects are assumed to be upright and level with the ground, without any tilting forward, backward, or sideways. This simplification means that only the yaw angle needs to be considered for orientation. Additionally, the camera intrinsic matrix is assumed to be known during both training and inference.

Among these 3 properties, the dimension and rotation are strongly related to visual appearance, which can be captured by a network [33], while 3D localization is a challenging problem because the accuracy of 3D localization requires precise depth estimation. Therefore, correctly estimating depth is crucial for monocular 3D object detection. The primary reason 3D localization is more challenging is the high variability in 3D location, ranging roughly from 1 to 60 meters, and the fact that visual appearance is not as useful as dimensional and rotational properties.

3D localization is mostly done by learning the projected 3D center (x_c, y_c) , and in literature, there are two different formulations in learning a projected 3D center: One is to directly predict it by learning a heatmap representation [34, 35, 17]. The other is to further decompose a projected 3D center into the center of 2D bounding box (x_b, y_b) and the offset vector $(\Delta x, \Delta y)$ with $(x_c = x_b + \Delta x)$ and $(y_c = y_b + \Delta y)$ [19, 18], following the CenterNet [36] formulation.

In these settings, the depth of the 3D projected center should also be learned to obtain the final 3D location (x, y, z) . For depth estimation, 4 different methods exist in the literature. The first method is direct depth estimation, which relies on the appearance of an object and its surrounding pixels to directly regress depth z . Utilizing the projected 3D center (x_c, y_c) and accurate depth estimation, the camera coordinate 3D location x and y are determined as:

$$x = \frac{(x_c - c_x)z}{f_x}, \quad y = \frac{(y_c - c_y)z}{f_y}, \quad (3.1)$$

where (c_x, c_y) represent the coordinate of the principle point, and (f_x, f_y) are the horizontal and vertical focal lengths, respectively. The second method is depth from height, which tackles the depth estimation by decoupling it as predicting the physical height h and pixel height h_p of an object via this formula:

$$x = \frac{f_y h}{h_p}. \quad (3.2)$$

After obtaining z , x and y are calculated using Eq. 3.1. The third method, as described in RTM3D [37], directly estimates the depth and refines it using geometric constraints that align the predicted 3D bounding box with its 2D projection, ensuring physical plausibility and consistency. The fourth method, PnP, uses geometric constraints to obtain the depth based on the *least squared method*. We will discuss this method in more detail in Section 4.4.5.

3.2 Base Detector

After carefully scanning the literature, we chose to develop our method on top of the MonoCon [19] detector due to its easy-to-extend, straightforward yet innovative design. Furthermore, we also chose MonoCon because its model architecture is view-independent, making it a suitable base detector for developing methods that achieve accurate object detection and localization from various perspectives. MonoCon learns well-posed 2D contexts projected from 3D bounding boxes as auxiliary learning tasks to help monocular detection.

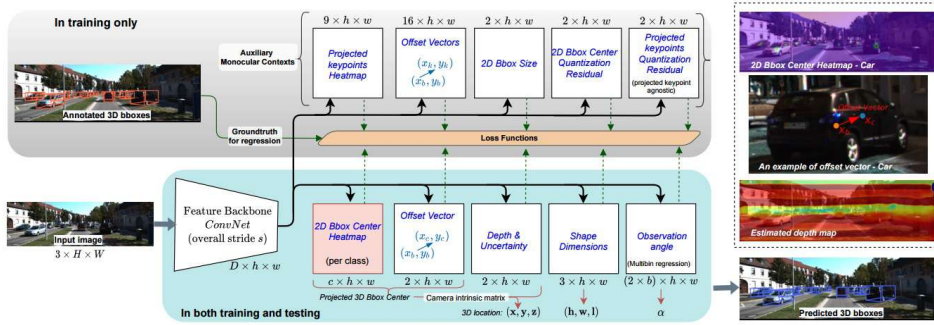


Figure 3.1: The Illustration of Used Base Detector MonoCon [19]

MonoCon employs lightweight regression heads for essential parameters (3D locations, shape dimensions, observation angles) and auxiliary contexts.

As shown in Figure 3.1, the proposed MonoCon method is simple by design, comprising these 4 components:

3.2.1 Feature Backbone

Given an input RGB image of dimension $3 \times H \times W$, a feature backbone $f(\cdot; \Theta)$ is used to compute the output feature map F of dimensions $D \times h \times w$ such as,

$$F = f(I_\Lambda; \Theta) \quad (3.3)$$

where Θ collects all the learnable parameters, D is the output feature map dimension n (e.g., $D = 512$), and h and w are determined by the overall stride/sub-sampling rate s in the backbone (e.g., $s = 4$). DLA network (DLA-34) [38] is used as a backbone, widely used in monocular 3D object detection.

3.2.2 3D Bounding Box Regression

MonoCon uses the anchor-offset formulation to learn the projected 3D bounding box center (x_c, y_c) in the image plane. First, the regression head is used to compute the class-specific heatmap H^b of dimensions $c \times h \times w$ for the 2D bounding box center (x_b, y_b) for each of the c classes (e.g., c representing the target classes such as car, pedestrian, cyclist, etc.). Then, the *offset regression head* is used to obtain offset vectors from the 2D bounding box center to the 3D projected center. Similarly, the depth and the shape dimensions are regressed respectively as follows,

$$Z_{1 \times h \times w} = \frac{1}{\text{Sigmoid}(g(F; \Theta^Z)[0] + \epsilon) - 1}, \quad (3.4)$$

$$\sigma_{1 \times h \times w}^Z = g(F; \Theta^Z)[1], \quad (3.5)$$

$$S_{3 \times h \times w}^{3D} = g(F; \Theta^{S^{3D}}), \quad (3.6)$$

where $g(F; \Theta^Z)$ directly estimates the depth and its uncertainty, and $g(F; \Theta^{S^{3D}})$ estimates the dimension of the 3D bounding box. σ^Z is used to model the heteroscedastic aleatoric uncertainty in the depth estimation as done in [39, 40, 34].

For the observation angle α , the multi-bin setting [41] is used. The angle range $[-\pi, \pi]$ is divided evenly into a predefined number of b non-overlapping bins (e.g., $b = 12$). The observation angle regression head is defined by,

$$A_{2b \times h \times w} = g(F; \Theta^A), \quad (3.7)$$

where the observation angle α is predicted by computing its bin index, $\alpha_i \in \{0, 1, \dots, 11\}$ from the first b channels and the corresponding angle residual, α_r , in the second b channels of A , together with proper conversions to ensure $\alpha \in [-\pi, \pi]$. The final angle α is calculated by first predicting the bin index α_i through classification, then computing the bin's center, and finally adding the predicted angle residual α_r from the regression step to the bin's center [19].

3.2.3 Decoding the Predicted 3D Bounding Box

Based on the peak in each channel of the heatmap H^b and thresholding with a threshold Γ , a set of 2D bounding box centers are detected for each class. Then the projected 3D center for the car is predicted by $(x_c, y_c) = (x_b + \Delta x_b, y_b + \Delta y_b)$. The depth prediction head predicts the corresponding depth directly, then the 3D location (x, y, z) can be trivially computed using the known camera intrinsic matrix. Similarly, the dimension (h, w, l) and the observation angle α can be predicted for the car. By using all these inferred parameters, the 3D bounding box is decoded.

3.2.4 The Auxiliary Context Regression

MonoCon suggests learning auxiliary tasks during training to improve generalization and, thereby, detection performance. Thus, it exploits four types of projection information from 3D bounding boxes. The first auxiliary task is *the heatmaps of the projected keypoints*. The heatmap of the 8 corner keypoints and projected center keypoint is learned as an auxiliary task. The second task is *the offset vectors for the 8 projected keypoints*. This task aims to learn the offset vector from the 2D bounding box center to 8 projected keypoints of the 3D bounding box. The third task is to predict 2D bounding box height and width, as done in CenterNet [36]. The last task is the *quantization residual of a keypoint location*, which models the residual of the 2D bounding box center (x_b, y_b) and that of the 9 projected keypoints (x_k, y_k) separately.

4 Methods

This section outlines the design and analytical techniques employed in this study. In Section 4.1, we defined three main problems that should be worked on to have generalized depth prediction. Sections 4.2, 4.3, and 4.4 provide detailed explanations of our approaches to solving these problems.

4.1 Problem Analysis

Achieving good depth estimations with different views is challenging due to the various installations and focal lengths of cameras. For example, vehicle-side cameras usually assume that the optical axis is parallel to the ground since the pitch angle is near zero. In contrast, the infrastructure-side cameras typically have large pitch angles, making most existing vehicle-side methods unsuitable for direct application. Therefore, predicting only the *azimuth* (*yaw*) angle while setting the *elevation* and *tilt* to zero, as done in the KITTI dataset, is not an optimal approach for accurate depth prediction across varying views, making it important to learn full rotation. In addition, the varying focal length of different cameras changes the visual appearance of the objects in the image, which may create scale-depth ambiguity, where the same object at two different depths can project to the same image. This scale-depth ambiguity is especially very common from an infrastructure-side view since the cameras and their installations are variable.

As mentioned in the works [42, 43, 7], the detection accuracy of objects can be improved by deducing complete shapes from vehicle keypoints. Although these methods require additional data, they give us the hint that the keypoints can also help improve detection accuracy in different views since they enhance geometric understanding. Furthermore, as demonstrated in MonoDDE [18], which does not use any extra data (aligning with our objective), the additional depth clues from keypoints can be used to improve depth estimation. However, MonoDDE is specifically tailored for car views, making it difficult to adapt to infrastructure-side views. By developing a more generic approach, depth estimation accuracy can be improved for any view using keypoints.

It is important to address the above-mentioned problems to have generalized monocular depth prediction.

4.2 Virtual Depth

It is crucial to address depth-scale ambiguity to achieve generalized depth prediction. Since the varying focal lengths affect the visual appearance, training the neural network with

different focal-length camera images harms the training process. Despite objects at different depths in the real world, they appear to be the same size in the image due to varying focal lengths. Providing these image samples to the neural network can negatively impact training, as the network may rely on appearance features to infer depth. Therefore, to have improved results in generalized depth prediction, it's necessary to establish a unified optimization target by removing the impact of focal length on depth. This can be achieved by incorporating focal length into the optimization process.

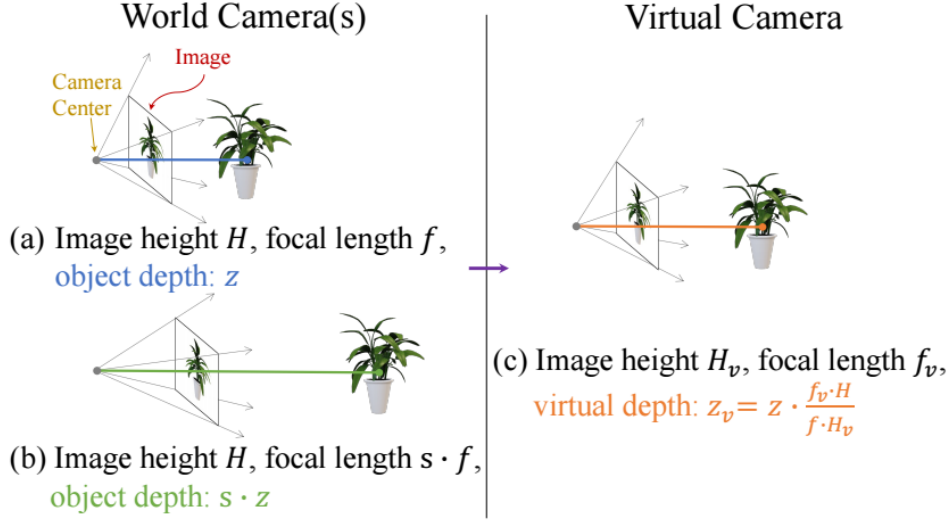


Figure 4.1: The illustration of virtual camera properties [5]

Inspired by the virtual camera idea [5], we implemented the *virtual depth* concept to address this issue, establishing it as a unified optimization target for our base detector, MonoCon. The virtual depth implementation projects the metric depth z to an intrinsic invariant target *virtual depth* z_v . It scales the depth using the known camera intrinsic so that the effective image size and focal length are consistent across the dataset, as illustrated in Figure 4.1. The effective image properties H_v (*virtual image height*) and f_v (*virtual focal length*) are both hyperparameters. We set these values to $f_v = 707.0493$, $H_v = 721.55$, resulting in a virtual depth scale factor of 0.9799. If z is the true metric depth, the unified optimization target virtual depth z_v can be calculated as:

$$z_v = z \cdot \frac{f_v \cdot H}{H_v} \quad (4.1)$$

We used z_v as an optimization target to train the neural network. Therefore, the network's predictions are also in virtual depth scale, which can be easily converted to metric depth z while decoding the predictions using the provided formula (4.1).

4.3 Predicting Full Rotation

Most detectors trained on the KITTI dataset assume the camera is parallel to the ground. Additionally, they assume that elevation and tilt angle are zero, and they only predict the azimuth angle. MonoCon also operates under these assumptions. While this approach works well with the car view, accurate 3D positioning of the target object is crucial for generalized monocular depth prediction, as learning the full rotation enables perspective-invariant learning for depth. In addition, learning full rotation enables the exploitation of depth information from keypoints using the PnP method, which is explained comprehensively in Section 4.4.5. This keypoint depth information may be useful for different views where detecting the center is challenging. Therefore, we have extended the rotation prediction to include the full rotation set θ, ϕ, ψ , which denotes yaw, pitch, and roll angles, respectively. The full rotation of the interested object is predicted in the form of a 6D continuous parameter since it is better suited for neural networks to regress compared to other forms of rotation, as shown in [44]. The predicted 6D continuous parameter mapped to 3x3 rotational matrix R_a . Similar to the [45], the 3D rotation matrix R_a is predicted in *allocentric* form since it aligned more closely with visual 2D evidence than the egocentric form. Identical egocentric rotations can appear significantly different when viewed from varying spatial locations because egocentric rotation uses the camera as a reference frame. In contrast, allocentric rotation does not have this limitation. This effect can be seen in the Figure 4.2.

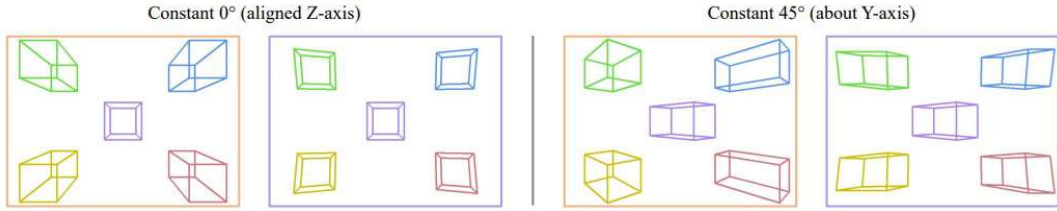


Figure 4.2: **Allocentric** and **Egocentric** Representations Under Constant Rotations [5]

Although the rotations are identical, the egocentric representation looks different as its spatial location changes, whereas the allocentric representation remains consistent regardless of location changes.

In inference time, allocentric rotation is converted to egocentric to compare with ground truth. This conversion can be done using the known camera intrinsics matrix. Let f_x, f_y, p_x, p_y be the known camera intrinsics, x_c, y_c the predicted 2D projected center as in Section 3.1, and $a = [0, 0, 1]$ be the camera's principal axis. Then $o = \text{norm} \left(\left[\frac{x_c - p_x}{f_x}, \frac{y_c - p_y}{f_y}, 1 \right] \right)$ is a ray pointing from the camera to x_c, y_c with angle $\alpha = \arccos(o)$. Using standard practices of axis-angle representations with an axis denoted as $o \times a$ and angle α , a matrix $M \in \mathbb{R}^{3 \times 3}$ can be computed, which helps form the final egocentric rotation matrix $R = M \cdot R_a$ [5].

4.4 Exploiting the Information in Keypoints for Depth Estimation

Most monocular 3D object detection methods primarily rely on object centers to estimate depth. While this method generally works well, incorporating keypoint information can significantly improve depth estimation accuracy. This is particularly useful in multi-view scenarios, where information at the center may be insufficient. By using keypoints, we can overcome this limitation and achieve more accurate depth predictions for the objects. An example of utilizing keypoint information is demonstrated in MonoDDE [8], where the authors enhanced depth estimation by aggregating keypoint depth estimations and center depth estimation for final depth measurement. Although effective, this approach was specifically tailored for the KITTI dataset and lacks generalization to other perspectives. To effectively utilize the information from keypoints, we introduced several modifications to our base method, MonoCon, and subsequently conducted various experiments.

4.4.1 Adding Auxiliary Keypoints

MonoCon employs 9 keypoints (1 center and 8 corner keypoints) to learn keypoint-related auxiliary tasks during training. These tasks include generating heatmaps for the projected keypoints, computing offset vectors for the 9 projected keypoints, and determining the quantization residual of keypoint locations. To explore whether additional keypoints could enhance training, we extended the number of keypoints by incorporating area center keypoints and edge center keypoints. The additional keypoints include the midpoint of each side of the bounding box (*center area keypoints*) and the midpoints of the bounding box edges (*edge center keypoints*), which can be seen in Figure 4.4.



Figure 4.3: All Keypoints After Addition: Default Keypoints (Blue), Edge Center Keypoints (Green), Area Center Keypoints (Red)

To evaluate the impact of incorporating additional keypoints, we tested our model under four different configurations:

1. No additional keypoints (9 keypoints in total).
2. Only area center keypoints (6 additional keypoints, $9+6=15$ keypoints in total).

3. Edge center keypoints (12 additional keypoints, $9+12=21$ keypoints in total)
4. Area center + edge center keypoints ($12+6=18$ additional keypoints, $9+18=27$ keypoints in total)

As we can see from Table 6.1, the additional keypoints did not significantly enhance the overall accuracy and introduced complexity to the auxiliary learning tasks, resulting in increased training time. Therefore, we decided to use the original 9 keypoints employed by MonoCon to develop our new features.

4.4.2 Keypoint-to-Center Offset Learning

MonoCon learns the Center to Keypoint (Center2Kpt) offset as an auxiliary task to determine keypoint locations in 2D. To extend this idea, we implemented keypoint-to-center offset learning, enabling each keypoint to learn its offset to the center. The main intuition behind this approach is that learning Keypoint to Center (Kpt2Center) offset may enhance object localization by improving the precision of keypoint locations. The Center2Kpt and Kpt2Center concepts are illustrated in Figure 4.4.

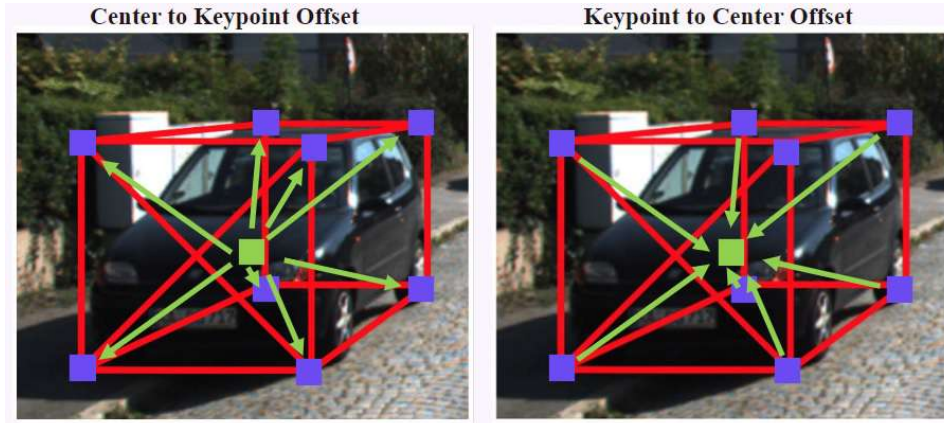


Figure 4.4: The Visualisations of Center-to-Keypoint and Keypoint-to-Center Concepts

To predict the Kpt2Center offset, we designed a head that generates a heatmap with separate channels for each keypoint. Therefore Given the input feature map, the Kpt2Center offset head predicts a heatmap with 18 channels, this includes 9 channels for the keypoints and 2 offset channels (x and y) for each keypoint. Due to the decrease in overall performance shown in Table 6.2, we decided not to include keypoint-to-center offset learning in our final approach.

4.4.3 Dense 3D Bounding Box Property Prediction Using Keypoints

In MonoCon, the 3D bounding box features such as depth, dimension, and orientation are predicted using only the center point of the object. To improve the prediction accuracy of

these 3D bounding features, we add keypoint-wise heads that can predict keypoint-wise depth, dimension, and orientation from the input feature map. After the detailed analysis, which can be found in Section 5.1, we decided to keep only the keypoint depth heatmap estimation and discard the allocentric and dimension keypoint heatmap prediction heads.

Keypoint Depth Prediction Head. MonoCon predicts the object’s depth directly by predicting a center depth map and then extracting the predicted depth by using predicted center indices. This method works well for car views, where object centers are easy to detect. However, it is not sufficient for generalized depth prediction from various perspectives. For instance, with infrastructure side cameras, corner keypoint features can sometimes be more useful than center keypoint features for predicting depth because detecting the center can be challenging. Furthermore, as explained in MonoDDE, exploiting keypoint information is also helpful for car view. Therefore, keypoints can be useful for both views, which is crucial for achieving generalized depth prediction. Based on this motivation, we decided to implement a keypoint depth prediction similar to the center depth prediction. This implementation uses keypoint indices instead of center indices.

Given feature map input, the keypoint depth prediction head generates a heatmap with shape $(B, 9 * 2, h/s, w/s)$, where B is the batch size, h , and w are the height and width of the image, and s is the feature map scaling (e.g., 4). The predicted heatmap contains the depth and uncertainty for each of the nine keypoint, resulting in 18 channels in the output.

To get the final keypoint depth predictions, we need to extract them from the predicted keypoint depth heatmap. We use keypoint indices for this operation. In training time, they are stored as ground truth, making the extraction easily achieved. After we gather the predicted keypoint depth heatmap, we obtain nine additional depth predictions from keypoints. Note that these keypoints correspond to projected 3D keypoints of the object, one for the center and eight for corner keypoints.

During inference, we do not have keypoint indices, which are necessary for obtaining keypoint depth predictions. To generate these indices, we first utilize the predicted keypoint heatmap, which serves as an auxiliary task in MonoCon. As mentioned in Section 3.2.4, this heatmap’s purpose is to learn and identify keypoint locations. We first apply a local maximum function to smooth the heatmap, then extract the top-k scores from it. Since it comprises nine channels (one for each keypoint), we ultimately obtain indices for all keypoints.

The general structure of this approach during inference is illustrated in Figure 4.5. Initially, an image is passed through the DLA-34 backbone to extract feature maps. These feature maps are then processed by two separate heads: the keypoint heatmap head and the keypoint depth heatmap head. The keypoint heatmap head generates keypoint heatmap predictions, from which the keypoint heatmap indices are obtained as explained. Simultaneously, the keypoint depth heatmap head produces keypoint depth predictions. Using the predicted keypoint indices, the depth values for the keypoints are extracted from the keypoint depth heatmap, resulting in the final nine depth and nine uncertainty predictions.

By employing the same heatmap extraction approach using predicted keypoint indices, we can also extract information from the predicted keypoint offset heatmap to calculate the

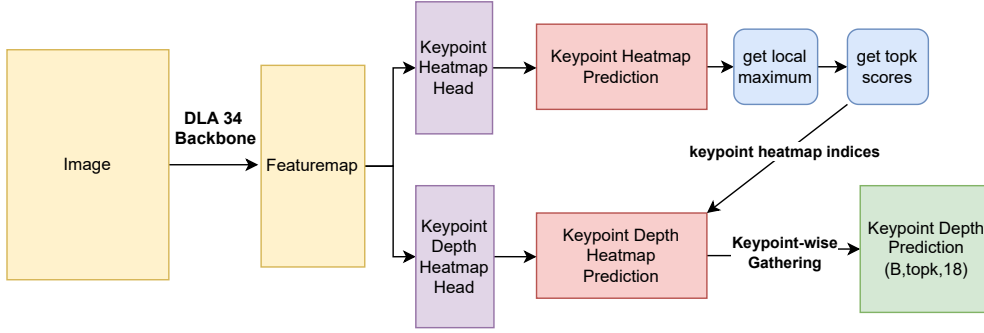


Figure 4.5: Extracting Keypoint Depth Predictions at Inference

keypoint locations in 2D. The 2D keypoints locations can be predicted with this formula:

$$\text{keypointloc}_x = \frac{\text{kpt}_x + \Delta_{\text{kpt}_x}}{s}, \quad \text{keypointloc}_y = \frac{\text{kpt}_y + \Delta_{\text{kpt}_y}}{s} \quad (4.2)$$

where s is the feature map stride, kpt_x and kpt_y are the predicted keypoint locations extracted from the predicted keypoint heatmap, and Δ_{kpt_x} and Δ_{kpt_y} are the predicted location offsets from the keypoint offset heatmap. By performing these calculations, we determine the 2D location and depth of the keypoints, which will be used to enhance our final results.

4.4.4 Associating Keypoint Predictions with Objects

In test time, it is straightforward to associate object predictions with ground truth objects by computing the Intersection over Union (IoU) between the predicted and ground truth 2D bounding boxes. Each ground truth object should have only one corresponding object prediction, ensuring a one-to-one association. However, associating heatmap-predicted keypoints with ground truth keypoints is more complex than the above approach. To understand the reason, consider the process of obtaining heatmap-predicted keypoints:

1. Keypoint indices are derived from the predicted keypoint heatmap using keypoint scores.
2. To extract these indices, the keypoint heatmap is first smoothed to identify local maxima, and then the top-k highest scores (e.g., 200) are selected.
3. Each keypoint has a separate channel in the heatmap, resulting in a keypoint indices tensor with the shape (B, N, topk) , where B is the batch size, N is the number of keypoints, and topk is the number of detectable objects in the input image.
4. These calculated keypoint indices are then used to extract information from the predicted keypoint depth heatmap.

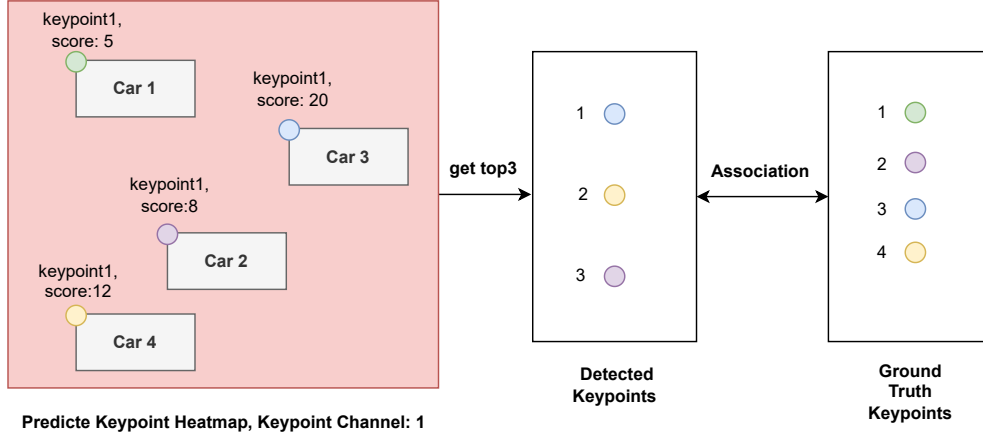


Figure 4.6: Loss of Object Association During Keypoint Detection

Since we extract the top-k scores from each heatmap’s keypoint channel to obtain keypoint location indices, the extracted keypoints lose the information about which object they belong to. This effect is better explained in Figure 4.6.

As we can see from Figure 4.6, extracting keypoints from the heatmap results in a loss of order information, meaning we cannot determine which object each keypoint belongs to. Therefore, it is necessary to associate these heatmap keypoint predictions with the ground truth keypoints.

To achieve accurate associations, we used the 2D Euclidean distance between heatmap-predicted keypoints and the ground truth keypoints. Since the predicted keypoint heatmap is agnostic to the ground truth objects, a ground truth keypoint can be associated with more than one predicted keypoint. Thus, we relaxed the association by using a radius approach. Given ground truth keypoint kpt_i , we associate this keypoint with every predicted keypoint within a radius r . We verify if the predicted keypoint is within the radius using the 2D Euclidean distance.

Objects farther from the camera appear smaller in the image, which causes issues if we use a static radius for each object. For example, $r = 15$ may work well for closer objects but can lead to many false positive associations for distant objects. To overcome this challenge, we dynamically set the search radius for each gt object based on its depth. The main idea of the dynamic radius is shrinking the search radius with increasing depth. To achieve this, we create a depth-radius mapping using the predetermined values: $depth_{max} = 56$, $depth_{min} = 2$, $radius_{max} = 20$, $radius_{min} = 5$ as shown in Figure 4.7.

Comparing the static radius search approach, the dynamic search radius is effective and reduces false keypoint detections, especially for far objects. Figure 4.8 illustrates the comparison between the static and dynamic radius approaches. From the figure, we can clearly see for the distant cars at the back; the search radius is shrunk, which produces a more feasible search radius for that object.

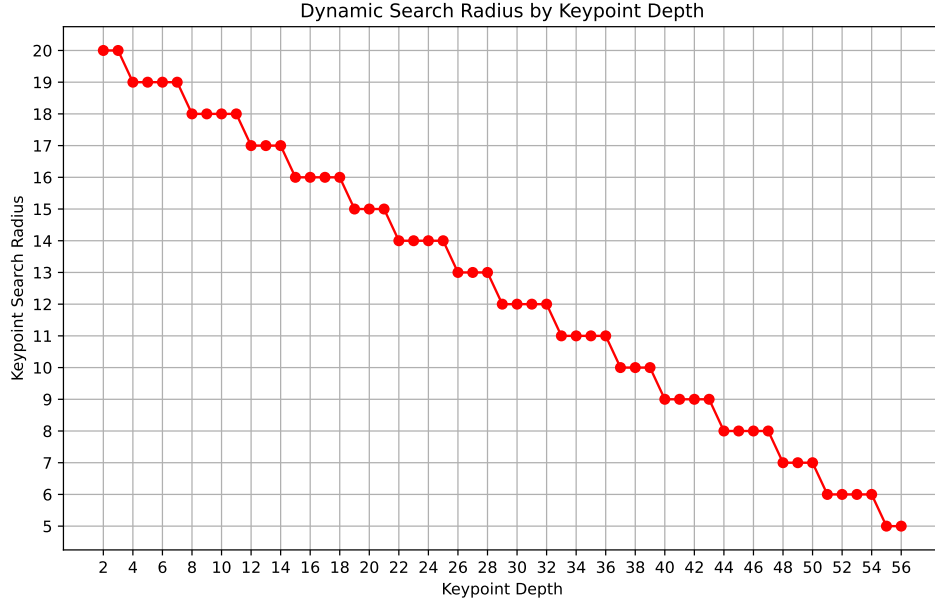


Figure 4.7: Dynamic Search Radius by Keypoint Depth



Figure 4.8: Comparison of Static (Left) and Dynamic(Right) Search Radius

4.4.5 Depth Estimation from Keypoints Using Perspective-n-Point

In 3D object detection, each instance is represented as a cuboid, which enables to use the of geometric constraints to obtain their 3D locations. Given the 3D keypoint position

$$P^o = (x^o, y^o, z^o)^T$$

under the object coordinate system, we can transform it to the camera coordinate system using rotation vector $\mathbf{R}$ and translation vector $\mathbf{T}$ as:

$$[x^c, y^c, z^c]^T = \mathbf{R} [x^o, y^o, z^o]^T + \mathbf{T}, \quad (4.3)$$

where $P^c = (x^c, y^c, z^c)^T$ represents the location of this 3D point under the camera coordinate system, and the rotation matrix R is given by:

$$R = R_z(\gamma)R_y(\beta)R_x(\alpha) \quad (4.4)$$

where:

$$\begin{aligned}
 R_x(\alpha) &= \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{pmatrix} \\
 R_y(\beta) &= \begin{pmatrix} \cos \beta & 0 & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{pmatrix} \\
 R_z(\gamma) &= \begin{pmatrix} \cos \gamma & -\sin \gamma & 0 \\ \sin \gamma & \cos \gamma & 0 \\ 0 & 0 & 1 \end{pmatrix}
 \end{aligned}$$

Combining these matrices, we get:

$$R = \begin{pmatrix} \cos \gamma & -\sin \gamma & 0 \\ \sin \gamma & \cos \gamma & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \cos \beta & 0 & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{pmatrix} \quad (4.5)$$

and for translation, we have:

$$\mathbf{T} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}. \quad (4.6)$$

Afterwards, given the camera intrinsic matrix $\mathbf{K}$, we can project $\mathbf{P}^c$ to a point in the 2D pixel coordinate system as (u, v) :

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} x^c \\ y^c \\ z^c \end{bmatrix}, \quad (4.7)$$

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_u \\ 0 & f_y & c_v \\ 0 & 0 & 1 \end{bmatrix}, \quad \lambda = z^c. \quad (4.8)$$

Here, the geometric relationships between any point in the object coordinate system and its corresponding pixel on the 2D imaging plane are described by Eqs. (4.3)- (4.8). In these equations, P^o is manually predefined, $\mathbf{K}$ is known, the 3D rotation matrix $\mathbf{R}$ and the keypoint 2D locations (u, v) are estimated by our network. Therefore, the 3D locations $\mathbf{T} = [x, y, z]^T$ contains the only variables to be computed. Since each 3D keypoint provides two geometric constraints, we can determine x , y , and z simultaneously using the least squares method if we have at least two keypoints [18]. Each object is represented with 8 corner keypoints, and we can calculate two separate depths (one for the pixel coordinate x and one for the pixel coordinate y) since each keypoint defines 2 geometric constraints. Therefore, this formulation allows us to obtain 16 additional depth predictions for each detected object using their predicted corner keypoints.

Predicting Keypoint 2D Locations. It is required to have keypoint 2D locations to calculate depths using PnP. In our approach, the 2D keypoint location can be predicted through three methods:

- Using the predicted center and Center2Kpt offset (*Center2Kpt 2D*),
- Using the predicted keypoint location and keypoint offset (*KptHeatmap 2D*),
- By combining predicted dimension, 3D center, and rotation with known calibration matrix to calculate projected keypoint 2D locations (*KptProjected 2D*).

After the experiments shown in Table 6.3, we decided to continue with the Center2Kpt offset to get predicted keypoint 2D locations, which can be easily obtained using the formula below:

$$loc_x = \frac{center_x + offset_x}{s}, loc_y = \frac{center_y + offset_y}{s}. \quad (4.9)$$

, where s in the equation is the feature map scale.

4.4.6 Learning Uncertainty for Keypoint-Based Perspective-n-Point Depth Predictions

As explained in Section 4.4.5, we can obtain two separate depth predictions for each keypoint, one for the keypoint’s x coordinate and one for the keypoint’s y coordinate. Henceforth, these two predicted depths will be referred to as the *formula predicted depth*. While obtaining formula-predicted depth with this approach is useful, small errors in either the x or y 2D coordinates can lead to significantly high depth prediction errors. This issue is particularly relevant in our case, as we rely on the predicted 2D locations to calculate the depth from the keypoints. To tackle this challenge, we trained the network to learn the uncertainty for each keypoint’s formula-predicted depth. Incorporating uncertainty in formula depth predictions can aid in selecting the most advantageous prediction during inference.

Therefore, we add a formula uncertainty prediction head to the model to learn the uncertainty for each formula depth prediction. The formula uncertainty head takes the feature map as input and outputs an uncertainty heatmap with 18 channels, meaning for each keypoint location (x,y), there are 2 channels in the output heatmap (one for x and one for y). The formula uncertainty for the corresponding keypoints is then extracted from the predicted heatmap using the keypoint indices. These extracted formula depth uncertainties are merged with the corresponding formula depth predictions to form depth-uncertainty pairs. Subsequently, the *Laplacian Aleatoric Uncertainty Loss* on predicted formula depth can be calculated via the equation (4.10).

$$L = 1.4142 \cdot e^{\log(-\tilde{\mu})} \cdot |\tilde{z} - z| + \tilde{\mu} \quad (4.10)$$

where $\tilde{z}$ is predicted formula depth, $\tilde{\mu}$ is predicted formula uncertainty and z is ground truth object depth. The formula depth calculation is performed without gradient computation using PyTorch’s no grad decorator, and we backpropagate the loss only through the formula uncertainty head, which results in better optimization for formula uncertainty estimation. The general diagram of learning formula depth uncertainty during training can be seen in Figure 4.9, where red arrows indicate the gradient flow direction.

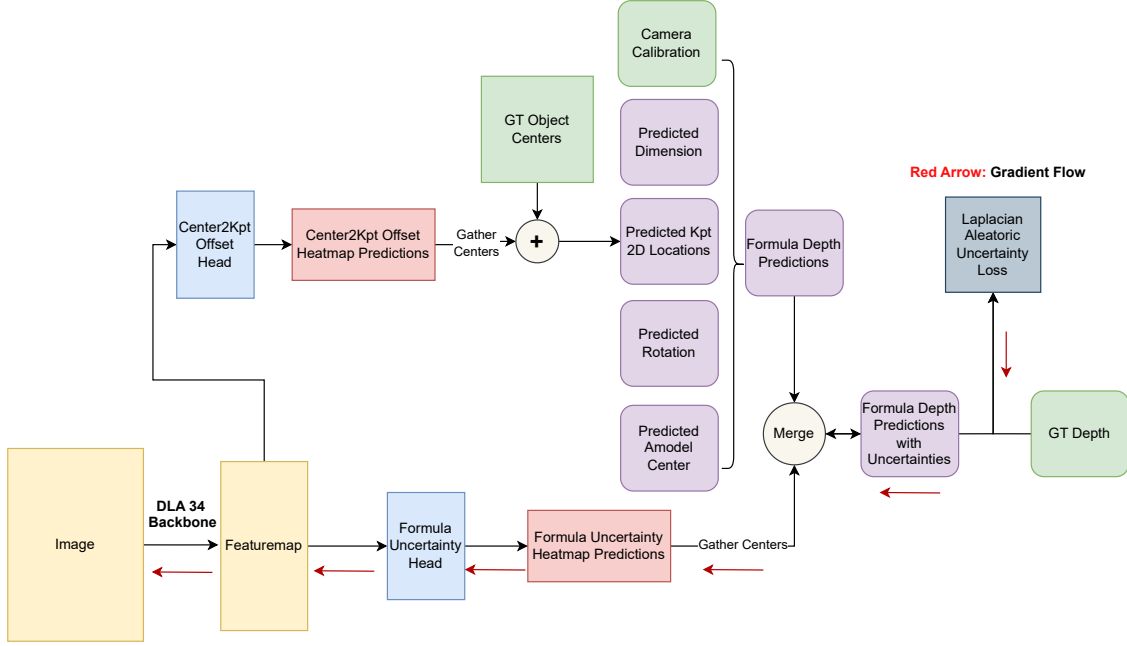


Figure 4.9: Learning Formula Depth Prediction Uncertainty

Calculating the formula depth predictions requires the Amodel center, dimension, rotation, and 2D keypoint locations. To predict the Amodel center, we used ground truth object centers and added the center offset (the last dimension of Center2Kpt prediction) from the predicted Center2Kpt offsets. Ground Truth (GT) object centers were used during training to mitigate the large localization errors that occurred initially, which would make training very difficult. During inference, centers were extracted from the predicted center heatmap and summed with the predicted center offset from the Center2Kpt to obtain the final Amodel center. Once the formula depth predictions are obtained, they are matched with the corresponding predicted uncertainties, and the loss is calculated using the ground truth depths.

4.4.7 Combining Dense Depth Estimations

For depth estimation, we utilize all depth predictions during inference, resulting in a total of 25 depth estimations: 1 from *direct depth*, 8 from *keypoint-heatmap depth*, and 16 from *formula depth estimation*.

For 3D object detection, each 3D bounding box is represented with a single depth. Therefore, we need to consolidate these depth estimations into one. By design, each of our depth estimations includes an uncertainty measure, enabling the use of the MonoDDE’s depth combination approach. We chose this approach because it helps mitigate the impact of outlier depth estimations from formula predictions, providing a robust method for combining depths.

This approach assumes that each depth estimation comes from separate Gaussian distributions.

Therefore, we define the distribution of a set $\mathbf{S} = \{s_i\}_{i=1}^N$, which contains N Gaussian distribution variables $s_i \sim \mathcal{N}(\mu_i, \sigma_i^2)$, as a new Gaussian distribution, because all the heads predict the depths of the same target. It is the weighted sum of $\{s_i\}_{i=1}^N$ and the weights $\{\omega_i\}_{i=1}^N$ are derived via the *weighted least squares method* [46]. We can define these weights as:

$$\omega_i = \frac{1/\sigma_i^2}{\sum_{j=1}^N 1/\sigma_j^2}. \quad (4.11)$$

Hence, the mean $\mu_{\mathbf{S}}$ and variance $\sigma_{\mathbf{S}}^2$ of $\mathbf{S}$ are calculated as:

$$\mu_{\mathbf{S}} = \sum_{i=1}^N \omega_i \mu_i, \quad \sigma_{\mathbf{S}}^2 = \sum_{i=1}^N \omega_i^2 \sigma_i^2. \quad (4.12)$$

In our case, set $\mathbf{S}$ contains 25 distributions. Since both $\mathbf{S}$ and its contained variables $\{s_i\}_{i=1}^N$ are treated as Gaussian distributions, we can filter out outliers based on the 3σ rule. The

Algorithm 1 Robust depth selection and combination

Require: Estimated depths $\{z_i\}_{i=1}^N$ and their corresponding variances $\{\sigma_i^2\}_{i=1}^N$

- 1: Initialize an empty set $\mathbf{S} = \emptyset$
- 2: Put z_k to $\mathbf{S}$ if $\sigma_k^2 = \min(\sigma_1^2, \sigma_2^2, \dots, \sigma_N^2)$
- 3: **while** True **do**
- 4: Update $\mu_{\mathbf{S}}$ and $\sigma_{\mathbf{S}}^2$ according to Eqs. (4.11)–(4.12)
- 5: $\mathbf{S}_{new} = \emptyset$
- 6: **for** $i = 1$ to N **do**
- 7: $\mathbf{S}_{new} \leftarrow \mathbf{S}_{new} \cup \{z_i\}$ if $z_i \in (\mu_{\mathbf{S}} - 3\sigma_{\mathbf{S}}, \mu_{\mathbf{S}} + 3\sigma_{\mathbf{S}})$ and $z_i \notin \mathbf{S}$
- 8: **end for**
- 9: **if** $\mathbf{S}_{new} \neq \emptyset$ **then**
- 10: break
- 11: **end if**
- 12: $\mathbf{S} \leftarrow \mathbf{S} \cup \mathbf{S}_{new}$
- 13: **end while**

Ensure: $\mu_{\mathbf{S}}$

robust depth combination algorithm starts by initializing an empty set $\mathbf{S}$ and adding the depth with the minimum variance to it. During the maximization step, $\mu_{\mathbf{S}}$ and $\sigma_{\mathbf{S}}^2$ are updated using Equations (4.11) and (4.12). In the expectation step, the depths that fall within the range $(\mu_{\mathbf{S}} - 3\sigma_{\mathbf{S}}, \mu_{\mathbf{S}} + 3\sigma_{\mathbf{S}})$ are added to the set $\mathbf{S}$. After $\mu_{\mathbf{S}}$ and $\sigma_{\mathbf{S}}^2$ stabilized, any depths outside the range $(\mu_{\mathbf{S}} - 3\sigma_{\mathbf{S}}, \mu_{\mathbf{S}} + 3\sigma_{\mathbf{S}})$ are considered outliers and are removed. This process ensures that $\mathbf{S}$ contains only the reliable depths. We use the final $\mu_{\mathbf{S}}$ as the combined depth z_c for further operations. The pseudo-code for this robust depth combination is provided in Algorithm 1.

5 Analysis

In this chapter, we will comprehensively analyze the internal mechanisms and outcomes of our proposed features using the KITTI dataset. Through a detailed examination, we aim to evaluate the efficiency and overall impact of our innovations. This analysis will highlight the strengths and weaknesses of our approach and provide insights into potential areas for improvement and further research. In Section 5.1, we analyzed how effectively the 3D feature keypoint heatmaps were learned. Next, in Section 5.2, we examined keypoint heatmap depth predictions and compared them with direct predictions. In Section 5.3, we assessed how well uncertainties were learned for formula depth predictions. Finally, in Section 5.4, we compared the combined depth estimations with direct depth estimations. For a detailed evaluation of the final results of our innovations from various viewpoints, please refer to Section 6.

5.1 Learning Keypoint Heatmaps for 3D Features

As mentioned in Section 4.4.3, we proposed learning 3D bounding box features using keypoints. To achieve this, we created three different heads that predict depth, dimension, and allocentric rotation heatmaps. Then, the depth, dimension, and allocentric rotation predictions are extracted from these heatmaps using keypoint indices.

After the insufficient preliminary experiment results for dimension, rotation, and depth heatmap prediction, we thought the keypoint dimension and rotation heads might hinder the optimization process. This issue likely arises because the targets for dimension and allocentric rotation are the same for each keypoint of the object, resulting in insufficient information for effective optimization. Therefore, we compared the model’s performance with and without them. As shown in Table 5.1, removing the keypoint dimension and allocentric

Table 5.1: Comparison of Evaluation Results with and without Keypoint Allocentric and Dimension Heads on KITTI

Experiment	Easy	Medium	Hard
MonoCon + Kpt Heads (depth+dim+alloc)	21.69 $\pm$ 0.37	16.72 $\pm$ 0.51	14.07 $\pm$ 0.56
MonoCon + Kpt Heads (only depth)	22.44 $\pm$ 0.88	17.08 $\pm$ 0.57	14.60 $\pm$ 0.27

heads positively affects performance. Therefore, we decided to add only the keypoint depth prediction head to the MonoCon.

We initially implemented the keypoint depth heatmap head with a separate channel for each keypoint. After several experiments, we discovered that using separate channels for

each keypoint negatively impacts the optimization process, leading to poor depth heatmap representations. Although learning separate channels for each keypoint works well for detecting keypoint gathering locations on the heatmap (keypoint indices), learning keypoint-wise 3D features (depth, keypoint offset, uncertainty, etc.) is more effective with a single shared channel. Therefore, we use a shared channel for all keypoints to learn the keypoint depth heatmap, which enhances the optimization process.

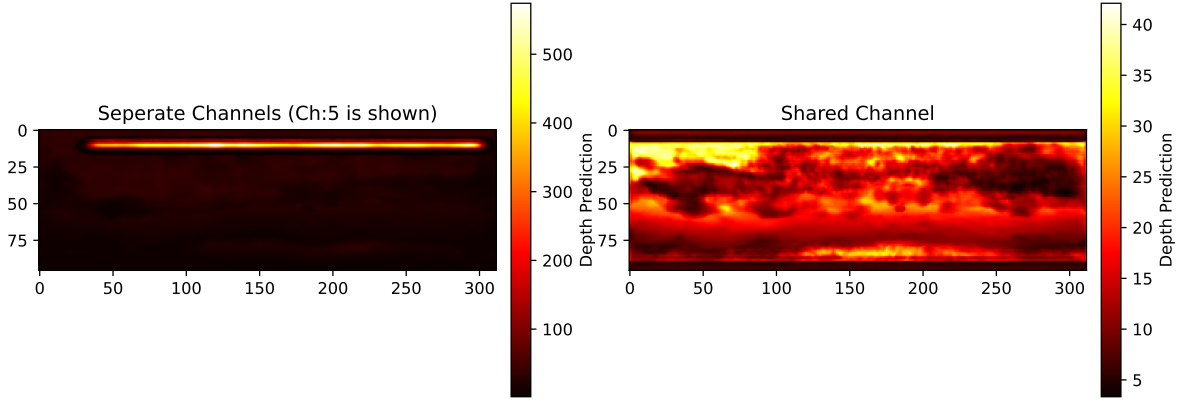


Figure 5.1: Comparison of Keypoint Depth Heatmap Prediction Distribution for Separate Channel-Shared Channel Implementation

In Figure 5.1, the left image is generated from the 5th keypoint’s internal depth heatmap prediction, while the right image shows the shared keypoint depth heatmap representation. As we can clearly see, using the shared channel for keypoint depth heatmap results in much better internal depth heatmap representations.

5.2 Keypoint Depth Heatmap Prediction Analysis

After we extracted depts from the keypoint depth heatmap, we have various depth predictions from different keypoints. However, not all the predicted keypoint depths are useful for the model, so a filter mechanism is required. To filter keypoint predictions, we calculate the 3D keypoint score for each one by multiplying its predicted keypoint heatmap score and keypoint heatmap depth uncertainty value. We then filter out keypoints using their 3D scores based on a certain threshold. After experimenting with different thresholds, we determined that 0.2 is the optimal value for detecting keypoints.

To analyze the keypoint depth heatmap predictions, we first match keypoint heatmap predictions with the predicted objects using a dynamic radius approach, then pair the ground truth objects with the predictions where the IoU is greater than 0.7. By applying this method to all predictions, we create a JSON file that contains pairs of objects, keypoint heatmap predictions, and their corresponding GT values. This JSON file is then used to generate statistics to evaluate the effectiveness of the predictions derived from the keypoint depth heatmap. To ensure a fair comparison of keypoint depth predictions and direct center predictions, we

first calculate the 3D location of a keypoint by its predicted 2D location and keypoint depth heatmap extracted depth. Then, utilizing the predicted dimensions and rotation, we convert this keypoint's 3D location to the center coordinates, which give us the predicted center depth. After this conversion, keypoint depth predictions are expressed as center depth, allowing us to fairly compare depth predictions of centers using direct and heatmap methods.

Kpt ID	Use Count	Detection%	Pred Max	Pred Min	Mean Error	Sigma Mean
kpt-1	5174	52.34	47.64	4.36	1.69	2.34
kpt-2	2322	23.49	49.05	2.35	2.79	1.5
kpt-3	2048	20.71	48.42	2.25	2.83	0.85
kpt-4	1915	19.37	48.71	2.46	2.79	0.99
kpt-5	1942	19.64	45.67	2.68	2.83	0.85
kpt-6	2009	20.32	47.89	2.64	2.93	0.98
kpt-7	2107	21.35	50.2	2.77	2.86	1.06
kpt-8	1721	17.41	47.5	1.51	2.87	1.14
kpt-9	2425	24.53	3.55	4.09	2.86	1.23

Table 5.2: Keypoint Statistics Including Use Count, Prediction Range, Mean Error, and Mean Sigma

Table 5.2 is created using all keypoint depth predictions on test data, and we can see that the depth range of keypoint heatmap depth predictions is reasonable. To illustrate how these mentioned keypoints appear on an image, we plotted Figure 5.2, where each keypoint is represented with a different color. These keypoints are highlighted in the table with corresponding colors for clarity and ease of understanding.



Figure 5.2: Locations of Nine Key Points Displayed on the Image

The model can detect the keypoint-1 and keypoint-9 best, and the lowest depth prediction error is in keypoint-1. The sigma value is calculated by:

$$\sigma = e^{-\tilde{\mu}} \quad (5.1)$$

where $\tilde{\mu}$ is the predicted uncertainty for the keypoint. As we can see in Equation 5.1, a higher sigma value means lower uncertainty, which means the model is more certain about the

prediction. The mean sigma value is calculated by averaging all predicted depth sigmas for the corresponding keypoint. Since the lowest error occurs at keypoint-1, we expect the model to be more certain about the prediction at keypoint-1, resulting in the highest sigma value there. This expectation is confirmed by our keypoint analysis.

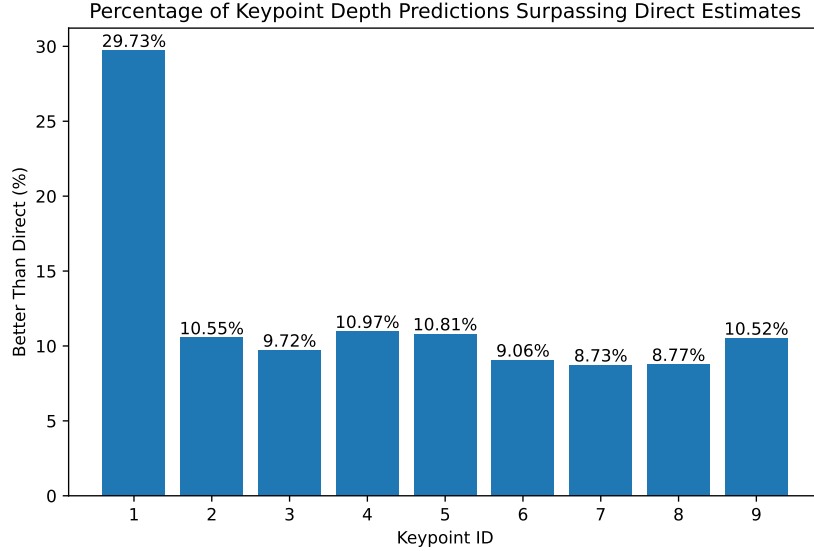


Figure 5.3: The Percentages of Keypoint Estimations That is Better Than Direct

For each keypoint, we calculate the percentage of instances where its prediction outperforms the direct depth prediction. These statistics are based on all test data.

Figure 5.3 illustrates the percentage of keypoints outperforming direct depth estimation. The figure suggests that while keypoint estimations can be beneficial, they must be carefully selected.

To see if the keypoint heatmap depth predictions can be useful for final depth estimation, we plotted the Mean Absolute Error (MAE) of direct and keypoint heatmap depth predictions. Since we can have multiple predictions from different keypoints, we have to gather these keypoint predictions into one to compare them with direct depth estimation. To generate the statistics in the plot, we discretize the GT depth ranges into consecutive bins (e.g., 1m, 2m, ... 50m), and for each GT bin, we aggregate the corresponding predictions and then calculate the MAE for that bin to determine the resulting error. For gathering, we used the mean and proposed MonoDDE robust depth combination method.

From Figure 5.4, we conclude MonoDDE outperforms the simple averaging of keypoint predictions, which shows combining depth estimations using predicted uncertainties via MonoDDE is effective. When comparing the direct depth estimation with MonoDDE-gathered keypoint depth estimation, it is observed that keypoint depth estimation has significantly lower error beyond 30 meters. This indicates the keypoint heatmap depth estimations can be useful, especially for far objects.

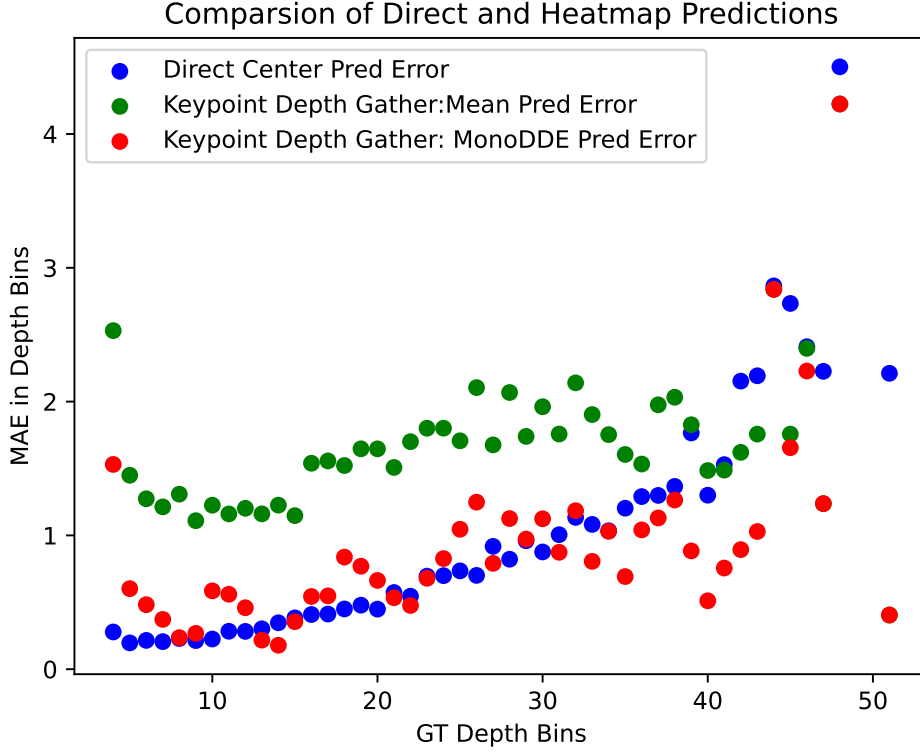


Figure 5.4: Comparison of Keypoint Heatmap Depth and Direct Depth Error

5.3 Learning Uncertainties for Formula Depth Predictions

Accurately learning uncertainties is crucial as they are used to obtain the final depth estimation. Direct depth estimation and keypoint heatmap depth estimation are already learning uncertainties using Laplacian Aleatoric Uncertainty Loss. For formula depth predictions, We created a separate uncertainty prediction head to learn uncertainty values. The output of this head is then combined with the estimated formula depth. This section analyzes whether the estimated uncertainties are accurate and useful for depth estimation.

If the learning formula uncertainties are working as expected, we would expect two outcomes: First, the uncertainty should increase with increasing depth. This is expected since objects farther from the camera should result in more uncertain model estimations. Second, the depth estimation error should increase with uncertainty. Higher uncertainty indicates the model is less confident about the depth prediction, which should lead to a higher error. To test these two facts, we calculated the relevant statistics on the KITTI dataset and plotted the results, as shown in Figure 5.5

The left plot is created similarly to Figure 5.4 by binning the GT depths and calculating the mean uncertainty for each bin. From this plot, uncertainty increases proportionally with depth between 10m and 50m, as we expected. However, the uncertainty decreases between 2m and 10m and between 50m and 52m, which is not an expected behavior. This may have

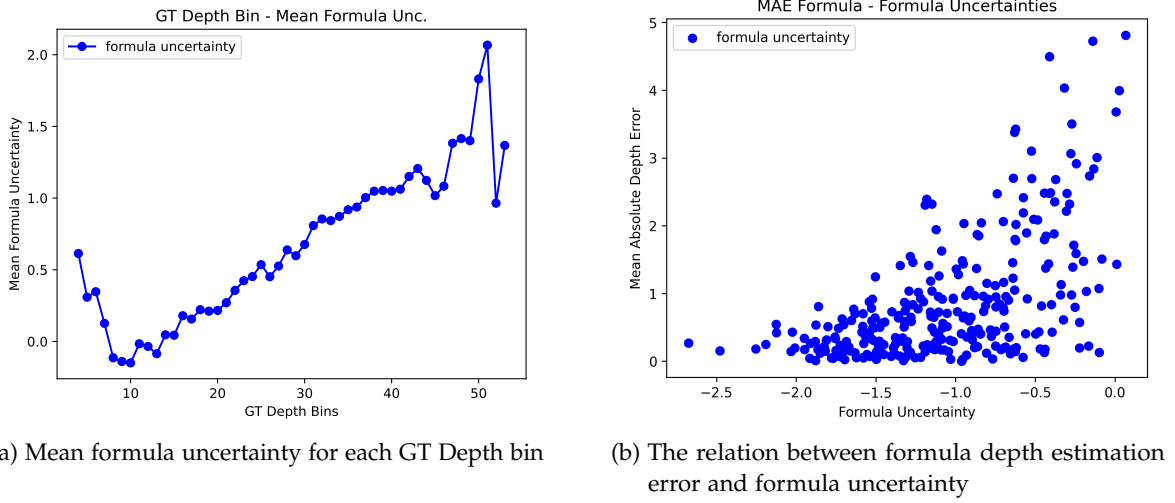


Figure 5.5: Formula Uncertainty Prediction Analysis

happened because the range between 10m and 50m has more data, making the measurements more reliable, while other depth ranges might suffer from less data. Additionally, the model may predict formula depth more accurately for medium-distance objects since the predictions used in the depth calculation (such as center, dimensions, and rotation) are better within this range. The right plot, on the other hand, shows the relationship between the predicted formula uncertainties and the corresponding errors in formula depth estimation. Looking at the error trend in this figure, we can conclude that the formula depth prediction error is proportionally increasing with the formula uncertainty.

The formula prediction method can produce very poor estimations, necessitating the incorporation of uncertainties for each depth estimate. To further evaluate the effectiveness of this learning uncertainty approach, we created Table 5.3, which presents the statistics for both formula-based and direct depth predictions.

Method	Pred Mean	Pred Std	Pred Max	Pred Min
Direct	22.25	10.27	49.86	3.87
Formula Gather: MonoDDE [18]	22.28	10.30	50.64	3.83
Formula Gather: Mean	147296.51	142756.10	13837031	-1355.94
Formula Gather: Median	22.64	14.24	454.26	-681.56

Table 5.3: Depth Prediction Statistics by Different Methods

As seen in table 5.3, the direct and formula MonoDDE methods provide consistent and reliable depth predictions, which shows us that learned uncertainties are useful. In contrast, the mean and median-based formula gathering methods are highly unreliable since they do not use the uncertainties for final depth estimation. Therefore, incorporating uncertainties in

formula depth estimation appears crucial to improve prediction reliability. To better compare the direct depth estimation and formula MonoDDE depth estimation, we plotted the MAE of these predictions on the same plot, as shown in Figure 5.6.

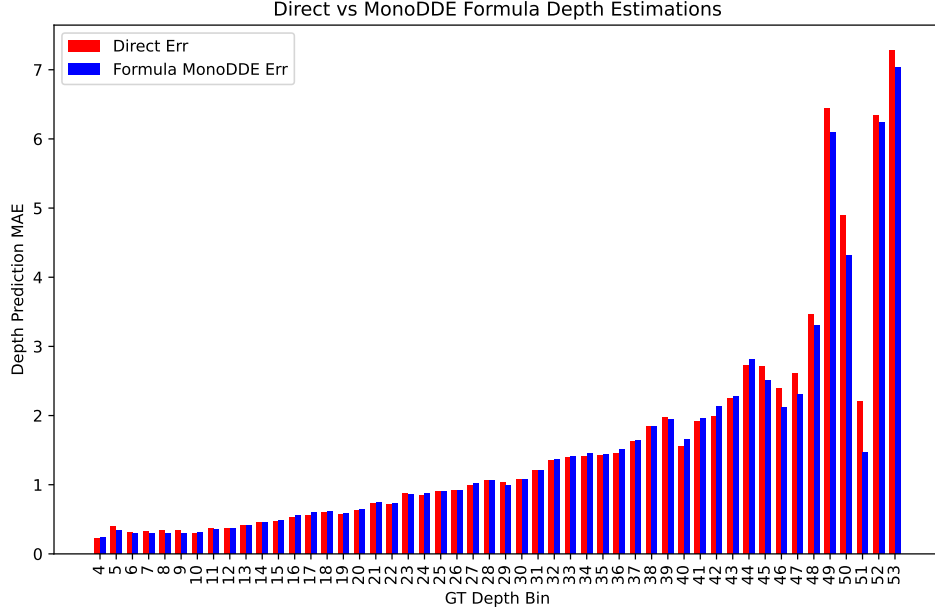


Figure 5.6: Comparison of MonoDDE Combined Formula Depth and Direct Depth Error

We can say the formula depth and direct depth estimations are close to each other, up to 45 meters. Beyond this point, the formula depth prediction performs slightly better than the direct estimation. From this plot and preceding analysis, it can be concluded that the formula predictions and their learned uncertainties are useful for our final depth prediction.

5.4 Combining Dense Estimations

Following a detailed analysis of the keypoint heatmap and formula depth estimation methods individually, this section focuses on the final ensemble of all methods. We refer to this final approach as *Dense Depth Estimation*, which integrates the Direct, Formula, and Keypoint Heatmap depth predictions using MonoDDE depth selection and combination approach. Since methods behave differently in various depth ranges, We divide the test data into bins called *Close*, *Medium*, and *Far* to have better insight into the final depth predictions. These bins are defined as follows:

$$\begin{aligned}
 \text{Close} : & \text{depth} \leq 15 \\
 \text{Medium} : & 15 < \text{depth} < 32 \\
 \text{Far} : & \text{depth} \geq 32
 \end{aligned}$$

The error statistics are calculated for these three different ranges.

Range	Estimation Method	Depth Error Statistics (m)					
		Count	Mean	Std	Median	Min	Max
Close	Direct	2645	0.36	0.35	0.26	0.0	5.35
	Dense (MonoDDE [18])	2645	0.37	0.37	0.27	0.0	2.64
Medium	Direct	3832	0.77	0.69	0.57	0.0	6.63
	Dense (MonoDDE)	3832	0.75	0.70	0.57	0.0	6.64
Far	Direct	1146	1.66	1.22	1.37	0.0	6.49
	Dense (MonoDDE)	1146	1.62	1.20	1.34	0.0	6.57

Table 5.4: Depth Prediction Error Statistics for Direct and Dense MonoDDE Ensemble Methods Across Ranges

As illustrated in Table 5.4, ensembling dense estimations with MonoDDE reduced the error in the medium and far range. For close range, the mean errors are comparable; however, using dense depth estimations decreased the maximum error from 5.35 to 2.64.

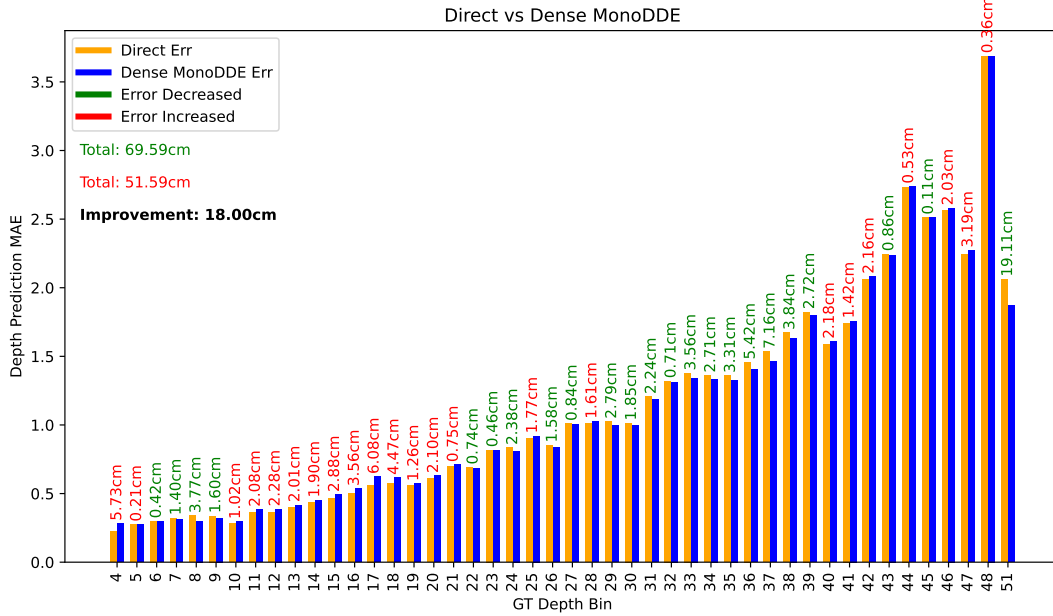


Figure 5.7: Comparative Analysis of Depth Estimation Error: Direct vs. Dense MonoDDE Method

Furthermore, in Figure 5.7, the MAE is reported for both Direct and Dense MonoDDE methods. It indicates the Dense MonoDDE method reduces the MAE across various GT depth bins. This reduction corresponds to an overall error decrease of 18.00 cm. Notably, in the final depth bin, the Dense MonoDDE method achieved a substantial error reduction of nearly 20 cm compared to the Direct method. Consequently, the Dense MonoDDE method offers a more reliable and accurate solution for depth prediction, particularly for distant objects, demonstrating advantages over the Direct method.

6 Experiments

In this section, we evaluate the performance of the proposed model for monocular depth estimation using the widely recognized and challenging KITTI and ROPE3D datasets. These datasets provide diverse and realistic scenarios from different perspectives, which is essential for assessing the effectiveness of our approach in 3D object detection for autonomous driving. We start by explaining the datasets and evaluation metrics used in our experiments. Following this, we present a detailed analysis of the proposed model through ablation studies, highlighting the contributions of each component to the overall performance.

6.1 Datasets

6.1.1 KITTI 3D

Our method is evaluated using the KITTI 3D object detection benchmark [26], which consists of 7481 training images and 7518 testing images. Since the annotations for the test set are not publicly available, we follow the approach in [47] and split the training data into 3712 images for training and 3769 images for validation. Based on pixel height and occlusion ratios, the KITTI dataset categorizes objects into three difficulty levels; Easy, Moderate, and Hard. For evaluation, we used the $AP_{3D} \mid R_{40}$ metric with IoU thresholds of 0.7 for cars, focusing primarily on the Moderate category.

6.1.2 ROPE3D

ROPE3D [31] is a comprehensive real-world benchmark for infrastructure-side 3D detection, featuring over 50,000 images and more than 1.5 million 3D objects in diverse scenes. The benchmark includes various settings, such as different cameras with varying mounting positions and diverse camera specifications. For evaluation metrics, we used $AP_{3D} \mid R_{40}$ and $Rope_{score}$, reporting the results for the Car and Big Vehicle classes.

6.2 Evaluation Metrics

Average Precision (AP) is a standard metric used to summarize the precision-recall curve, representing the trade-off between the model’s precision and recall. $AP_{3D} \mid R_{40}$ is a special type of AP that measures the average precision of a 3D object detection algorithm using 40 recall points. This metric is crucial for understanding how well the model balances precision and recall across different thresholds. AP_{70} measures the average precision of an object detection algorithm at a 70% IoU threshold, indicating stricter localization accuracy, while

AP_{50} does so at a 50% IoU threshold, allowing more tolerance in object localization.

Average Orientation Similarity (AOS) is a metric to evaluate the accuracy of the orientation of detected objects. The overall AOS is calculated by averaging the orientation similarities, which is computed as the cosine of the angle difference between the detected and ground truth orientations [31].

Average Ground Center Similarity (ACS) is the distance between objects on the ground plane indicates the risk of collision, calculated as:

$$ACS = \frac{1}{|D|} \sum_{s \in D} \left(1 - \min \left(1, \frac{\Delta_s^c}{C_s} \right) \right) \quad (6.1)$$

where D is the set of true positive samples, C_s is the norm of the GT ground center, and Δ_s^c is the Euclidean distance between the predicted ground center and the GT ground center of sample s . $|D|$ is the total number of true positive objects [31].

Average Area Similarity (AAS) is the ground occupancy of the prediction w.r.t the GT in terms of area, Δ_s^A is the absolute area difference and A_s is the GT area [31].

$$AAS = \frac{1}{|D|} \sum_{s \in D} \left(1 - \min \left(1, \frac{\Delta_s^A}{A_s} \right) \right) \quad (6.2)$$

Average Ground Points Similarity (AGS) is the similarity measure between estimated and GT ground points [31], calculated as:

$$AGS = \frac{1}{|D|} \sum_{s \in D} \left(1 - \min \left(1, \frac{1}{K} \sum_{g=0}^{K-1} \frac{|s_g - \hat{s}_g|}{|\hat{c}|} \right) \right) \quad (6.3)$$

Where $\hat{s}_g$ and s_g are the g -th predicted and GT ground points of sample s , respectively. $K = 4$ is the total number of ground points [31].

Rope_{score} is part of the evaluation metrics used in the Rope3D dataset, which is designed for roadside perception in autonomous driving. Assume $S = \frac{(ACS+AOS+AAS+AGS)}{4}$, then the $Rope_{score}$ is calculated by reweighting the 3D AP and the proposed similarity metrics with $\omega_1 = 8$ and $\omega_2 = 2$ [31]. The $Rope_{score}$ calculated by using AP_{50} and AP_{70} is called $Rope_{50}$ and $Rope_{70}$ respectively.

$$\mathbf{Rope}_{score} = \frac{\omega_1 * AP + \omega_2 * S}{\omega_1 + \omega_2} \quad (6.4)$$

6.3 Implementation Details

We conducted preliminary experiments on a subset of the KITTI by using 50% of the data and trained the model for 140 epochs to assess the benefits of the experimental additions. These smaller-scale experiments in KITTI allowed us to quickly assess various modifications. For the ROPE3D dataset, we used the configuration "train compressed" to evaluate the final proposed model. The "train compressed" configuration included 8,000 images for training and 4,676 for validation. For KITTI, we used 100% of the data for final model evaluations,

extending training to 200 epochs. In ROPE3D, each experiment was trained for 60 epochs. For KITTI and ROPE3D, we applied Photometric Distortion, Random Shifts and Crops, Random Horizontal and Vertical Flips, Random Rotations, and Random Scaling augmentations.

Following the original MonoCon training schedule, we employed the AdamW optimizer with $(\beta_1, \beta_2) = (0.95, 0.99)$ and a weight decay of 0.00001. The initial learning rate was set to 2.25×10^{-4} . After training, the auxiliary components introduced by the original MonoCon detector were discarded.

During inference, we first decoded the predictions to obtain dense depth estimations. These dense depth estimations were combined using the MonoDDE robust depth combination approach [18]. Subsequently, we updated the gravity center based on the newly calculated depths and performed the evaluation using these updated gravity centers.

6.4 Qualitative Results

6.4.1 Adding Auxiliary Keypoints

We evaluated this method on the KITTI dataset by conducting five runs for each experiment, with each run comprising 140 epochs and utilizing 50% of the data, as shown in Table 6.1. The table entries indicate the mean and standard deviation (mean $\pm$ std) of the experiment runs.

Table 6.1: Auxiliary Keypoints Evaluation Results for AP40 IOU 0.7 CARS.

Experiment	Easy	Medium	Hard
Default	19.85 $\pm$ 0.92	14.45 $\pm$ 0.69	12.25 $\pm$ 0.35
Default+Area Centers	19.53 $\pm$ 0.51	14.60 $\pm$ 0.36	12.44 $\pm$ 0.24
Default+Edge Centers	19.56 $\pm$ 1.08	14.78 $\pm$ 0.60	12.17 $\pm$ 0.22
Default+Edge+Area Centers	18.80 $\pm$ 0.87	13.46 $\pm$ 0.34	11.81 $\pm$ 0.51

Although the default+area centers and default+edge centers settings increased the overall AP score for Medium, they decreased the settings Easy and Hard, and the increase was insignificant. Furthermore, adding keypoints introduced complexity to the system, making it harder to learn the keypoint depth heatmap. Therefore we decided to continue using the original nine keypoints.

6.4.2 Keypoint2Center Offset Learning

To evaluate the effectiveness of this approach, we conducted experiments on KITTI under the same settings as described in Table 6.1, with the results presented in Table 6.2.

Adding Kpt2Center decreased the performance. The Kpt2Center head predicts a separate channel for each keypoint, which would be the cause of the performance decrease. When predicting keypoint offsets in separate channels, the gradients for each channel might not

Table 6.2: Keypoint2Center Offset Learning Evaluation Results for AP40 IOU 0.7 CARS

Experiment	Easy	Medium	Hard
Only Center2Kpt Learning	19.85 $\pm$ 0.92	14.59 $\pm$ 0.69	12.54 $\pm$ 0.35
Center2Kpt + Kpt2Center Learning	19.75 $\pm$ 0.99	14.45 $\pm$ 0.60	12.25 $\pm$ 0.21

be strong enough to provide clear direction for the overall model optimization. In addition, predicting offsets in separate channels might lead to spatial inconsistencies across channels.

6.4.3 Predicting 2D Keypoint Locations

To determine which method performs better for 2D keypoint location estimation, we calculate the 2D Euclidean distance between the predicted keypoint locations and the GT, as shown in Table 6.3.

Keypoint ID	Center2Kpt 2D	KptHeatmap 2D	KptProjected 2D
kpt1	5.12 $\pm$ 3.66	6.68 $\pm$ 8.02	6.97 $\pm$ 12.52
kpt2	5.92 $\pm$ 4.00	7.23 $\pm$ 10.89	7.88 $\pm$ 13.17
kpt3	5.06 $\pm$ 3.54	6.12 $\pm$ 7.87	6.31 $\pm$ 8.45
kpt4	5.71 $\pm$ 3.79	6.21 $\pm$ 8.17	6.41 $\pm$ 8.64
kpt5	6.10 $\pm$ 3.93	6.35 $\pm$ 8.35	6.55 $\pm$ 9.21
kpt6	6.22 $\pm$ 4.07	7.07 $\pm$ 9.91	7.32 $\pm$ 15.59
kpt7	5.16 $\pm$ 3.62	7.26 $\pm$ 11.48	7.39 $\pm$ 15.76
kpt8	6.01 $\pm$ 4.03	7.64 $\pm$ 12.15	7.82 $\pm$ 12.44
kpt9	4.41 $\pm$ 4.27	4.65 $\pm$ 3.41	4.41 $\pm$ 4.28

Table 6.3: Comparison of 2D Euclidean Distances Between Ground Truth and Various Keypoint Estimation Methods

As we can see in Table 6.3, the Center2Kpt method outperforms other methods on all keypoints. Therefore, we used Center2Kpt’s 2D location estimations for all methods requiring keypoint 2D locations.

6.5 Ablation Studies

To understand how much improvement each component provides, we performed ablation studies on the KITTI validation set for the Car category and the ROPE3D validation set for the Car and Big Vehicle categories.

6.5.1 Virtual Depth and Full Rotation

To evaluate the effects of the proposed virtual depth and learning full allocentric rotation, we conducted 5 experiments with the original MonoCon and 5 with MonoCon enhanced by virtual depth and allocentric rotation. We reported AP_{70} and AOS metrics for the Car class in KITTI, and AP_{50} , AP_{70} , $Rope_{50}$, and $Rope_{70}$ metrics for the Car and Big Vehicle classes in ROPE3D. The results for KITTI are shown in Table 6.4. To calculate the statistics, we conducted 5 experiments on 100% of the KITTI dataset, each running for 200 epochs. For ROPE3D, the ablation study results for the Car class are presented in Table 6.5, while the results for the Big Vehicle class are shown in Table 6.6. The ablation study for ROPE3D is conducted with 60 epochs on 8000 data.

Table 6.4: KITTI Ablation Study Results for Virtual Depth and Allocentric Rotation on Car Category

Method	AP40 %70			AOS		
	Easy	Medium	Hard	Easy	Medium	Hard
MonoCon	23.05 $\pm$ 0.65	17.23 $\pm$ 0.74	14.92 $\pm$ 0.36	45.39 $\pm$ 1.07	41.37 $\pm$ 0.58	37.37 $\pm$ 0.46
Monocon+Virtual Depth+Allocentric Rot.	23.68 $\pm$ 0.30	17.60 $\pm$ 0.55	15.01 $\pm$ 0.37	45.04 $\pm$ 0.99	41.04 $\pm$ 0.44	37.08 $\pm$ 0.32

Table 6.5: Rope3D Virtual Depth and Allocentric Rotation Ablation Study for Car

Method	AP40 %50	AP40 %70	Rope%50	Rope%70
MonoCon	0.257 $\pm$ 0.007	0.040 $\pm$ 0.05	0.395 $\pm$ 0.006	0.186 $\pm$ 0.004
MonoCon+Virtual Depth+Allocentric Rot.	0.267 $\pm$ 0.034	0.045 $\pm$ 0.09	0.403 $\pm$ 0.028	0.189 $\pm$ 0.010

Table 6.6: Rope3D Virtual Depth and Allocentric Rotation Ablation Study for Big Vehicle

Method	AP40 %50	AP40 %70	Rope%50	Rope%70
MonoCon	0.161 $\pm$ 0.026	0.002 $\pm$ 0.002	0.301 $\pm$ 0.021	0.141 $\pm$ 0.004
MonoCon+Virtual Depth+Allocentric Rot.	0.170 $\pm$ 0.032	0.003 $\pm$ 0.003	0.314 $\pm$ 0.025	0.142 $\pm$ 0.003

From the Rope3D results, we can conclude that adding virtual depth and allocentric rotation enhanced the detection accuracy in all metrics. They increased the detection accuracy of both the Car and Big Vehicle classes in ROPE3D by addressing issues arising from different camera installations and focal lengths. This supports our hypothesis that virtual depth and full allocentric rotation help detect objects in varying camera installation and focal length settings. The results for the KITTI dataset are also improved, indicating that virtual depth and learning full rotation improved the accuracy in both the car view and infrastructure side view. The AOS results for KITTI demonstrate that our method can learn full rotation with

an accuracy nearly as high as that for alpha learning, indicating that the proposed method performs well in KITTI, even with full rotation learning. These results show that virtual depth and learning full allocentric rotation are adaptable to various viewing conditions.

6.5.2 Dense Depth Estimation

We conducted experiments to evaluate the proposed dense depth estimation method by comparing models with and without using dense depth during inference. To see the effects of each component, we compared these settings: only direct depth estimation, direct depth combined with keypoint depth estimation, direct depth combined with formula depth estimation, and lastly, direct depth combined with formula and kpt heatmap depth estimation. Using only direct depth means that we do not use the predicted depth results in inference time, which means the keypoint depth and formula depth are only learned as auxiliary. When we say Dense(direct + formula depth), we use the MonoDDE combination approach in inference time by combining direct and formula depth. After we get the combined depth, we calculate the new gravity centers with these combined depths and evaluate the results. This is valid for all methods called Dense in the tables.

Table 6.7: KITTI Car Category Ablation Study for Dense Depth Estimation

Method	AP40 %70		
	Easy	Medium	Hard
Direct Depth	23.04 $\pm$ 0.96	17.29 $\pm$ 0.50	14.71 $\pm$ 0.39
Dense(KptHeat+ Direct Depth)	23.09 $\pm$ 0.95	17.32 $\pm$ 0.47	14.81 $\pm$ 0.44
Dense(Formula+ Direct Depth)	23.09 $\pm$ 0.92	17.31 $\pm$ 0.48	14.75 $\pm$ 0.34
Dense(KptHeat+ Formula + Direct Depth)	23.11 $\pm$ 0.87	17.37 $\pm$ 0.40	14.89 $\pm$ 0.30

Table 6.8: ROPE3D Car Category Ablation Study for Dense Depth Estimation

Method	AP40 %50	AP40 %70	Rope%50	Rope%70
Direct Depth	0.236 $\pm$ 0.010	0.031 $\pm$ 0.003	0.378 $\pm$ 0.008	0.178 $\pm$ 0.002
Dense(KptHeat+ Direct Depth)	0.237 $\pm$ 0.014	0.033 $\pm$ 0.003	0.379 $\pm$ 0.011	0.179 $\pm$ 0.003
Dense(Formula+ Direct Depth)	0.236 $\pm$ 0.011	0.032 $\pm$ 0.004	0.379 $\pm$ 0.009	0.178 $\pm$ 0.003
Dense(KptHeat+ Formula + Direct Depth)	0.238 $\pm$ 0.013	0.033 $\pm$ 0.002	0.381 $\pm$ 0.011	0.180 $\pm$ 0.002

For the KITTI dataset, combining all depth estimations improved detection accuracy in the Car class. This shows us that integrating dense depth estimation during inference enhances detection accuracy, especially for medium cases.

Table 6.9: ROPE3D Big Vehicle Category Ablation Study for Dense Depth Estimation

Method	AP40 %50	AP40 %70	Rope%50	Rope%70
Direct Depth	0.169 ± 0.013	0.005 ± 0.003	0.313 ± 0.010	0.144 ± 0.003
Dense(KptHeat+ Direct Depth)	0.174 ± 0.015	0.006 ± 0.002	0.318 ± 0.011	0.145 ± 0.002
Dense(Formula+ Direct Depth)	0.168 ± 0.013	0.005 ± 0.002	0.312 ± 0.009	0.143 ± 0.003
Dense(KptHeat+ Formula + Direct Depth)	0.170 ± 0.014	0.006 ± 0.002	0.314 ± 0.010	0.144 ± 0.003

Examining the results for ROPE3D in Tables 6.8 and 6.9, we can draw the following conclusions: Using dense estimation with all depth methods most improves detection in the Car category. The combination of Dense(Direct + Formula) also enhances accuracy. However, the Dense(Direct + KptHeat Depth) method shows the least performance improvement. For the Big Vehicle Dataset, all combination methods, except Dense(Direct + Formula Depth), improved final detection results. The best improvement was observed with Dense(Direct + KptHeat Depth), suggesting that learning keypoint heatmaps is beneficial for detecting big vehicles. The Dense(Direct + Formula Depth) method decreased accuracy compared to the Direct Depth method. This decrease might be due to incorrect rotation and dimension estimates propagating into the formula depth predictions, as these predictions rely on those estimates for depth calculation. For detailed analysis, discussions, and research outcomes, please refer to Chapter 7.

7 Discussion

This thesis explored three methods for monocular depth estimation to achieve generalized depth prediction for autonomous driving. The first method, virtual depth, is proposed to address depth-scale ambiguity. This method incorporates the focal length into the optimization process, establishing a unified optimization target called virtual depth. The second method proposed learning full rotation to enable perspective-invariant learning for depth. The 3×3 allocentric rotation matrix is learned using a 6D continuous parameter.

From the ablation results presented in Section 6.5.1, it is evident that combining virtual depth and allocentric full rotation significantly improved detection accuracy in the ROPE3D dataset for both the Car and Big Vehicle classes. These improvements are ascribed to the methods' ability to address problems resulting from different camera installations and focal lengths, thereby supporting our hypothesis that virtual depth and full allocentric rotation function well in diverse camera configurations.

The results for the KITTI dataset corroborate these findings. Virtual depth and full rotation learning improved detection accuracy for both car and infrastructure side view scenarios. The AOS results for KITTI demonstrate that our method can learn full rotation with an accuracy nearly equivalent to that of only learning yaw angle α , indicating the robustness of the proposed method even when learning full rotation.

These findings underscore the adaptability of virtual depth and allocentric rotation across various viewing conditions, showing their effectiveness in generalized monocular depth prediction for autonomous driving. Integrating virtual depth and learning allocentric rotation addresses critical challenges in depth estimation, leading to more accurate and reliable 3D object detection.

The third method we proposed involves exploiting keypoints, motivated by the idea that their spatial information and depth clues can enhance geometrical understanding and improve depth prediction. With this motivation, first, we extended the number of keypoints by incorporating area center keypoints and edge center keypoints to explore whether additional keypoints could enhance training. However, the additional keypoints did not significantly enhance overall accuracy and introduced complexity to the auxiliary learning tasks, resulting in increased training time. This may have occurred because the additional keypoints created more noise than useful data, which affected the model's learning process. Inspired by the Center2Kpt idea, we also implemented Kpt2Center offset learning, enabling each keypoint to learn its offset to the center. The main intuition behind this approach is that learning Kpt2Center offsets may enhance object localization by improving the precision of 2D keypoint locations and projected 3D centers of the objects. However, we decided not to include it in our final approach due to the decrease in overall performance shown in Table 6.2. This decrease may have occurred because introducing Kpt2Center offset learning added unneces-

sary complexity, thus reducing its efficiency and accuracy. Defining keypoints relative to the center could also introduce ambiguity, especially in objects with unclear centers, potentially leading to less accurate localization. Furthermore, the Kpt2Center offsets might have caused redundant information already captured by Center2Kpt learning, therefore not useful and only adding complexity to the model.

To further exploit keypoint information, we propose using depth clues from keypoints to improve the final depth prediction of the 3D bounding box. The central idea is that keypoints can be useful for both car and infrastructure views, which is crucial for achieving generalized depth prediction. Based on this motivation, we implemented a keypoint depth prediction head, which can learn the depth and uncertainties of the keypoints. We showed that learning keypoint depths using a shared heatmap is better for optimization.

From the analysis in Section 5.2, we can conclude that learning keypoint heatmaps is particularly beneficial in settings where detecting the center is challenging, such as with distant objects. Additionally, the ablation study in Section 6.5.2 shows using the estimations from the keypoint depth map during inference improved detection accuracy for the Big Vehicle class in Rope3D. However, for the Car category in ROPE3D and KITTI, the keypoint depth heatmap predictions during inference did not significantly improve detection accuracy. This discrepancy may arise due to several reasons.

Firstly, the inherent variability and higher density of cars in urban environments, as seen in the KITTI dataset, can result in overlapping bounding boxes, complicating the task of accurately predicting depth from the keypoint depth heatmap. Also, the simpler geometry and more consistent shapes of bigger vehicles might allow the keypoint-based depth prediction to perform more effectively, as the keypoint patterns are easier to learn and generalize. Lastly, the quality and resolution of the keypoint annotations could be more precise for larger vehicles, providing a clearer signal for the depth prediction model. Hence, it can be said that while keypoint depth heatmaps are a powerful tool, their effectiveness can vary based on object class and dataset characteristics. Better association algorithms can be explored to improve the effectiveness of keypoint heatmap depth predictions since more accurate associations result in more precise keypoint depth predictions for the objects.

In addition to keypoint heatmap depth prediction, we estimated extra keypoint depths using the perspective-n-point method. This method incorporates predicted dimensions, full rotation, and a model center to estimate depth, which we call the formula depth. We also learned uncertainty values for our formula depth estimations to robustly filter the outlier depth values predicted by it. Our general analysis of formula depth uncertainty predictions, found in Section 5.3, shows the formula uncertainties behaved as expected, except for car ranges below 10m and above 50m. This discrepancy may be due to the higher data density between 10m and 50m, making measurements more reliable, while other ranges suffer from less data. However, as discussed in Section 5.3, the learned formula depth uncertainties remain useful for final depth predictions because they help to eliminate outlier predictions.

The ablation study in Subsection 6.5.2 shows that using formula depth predictions during inference improved detection accuracy for the Car class in KITTI and ROPE3D, though the improvement was not significant. For the Big Vehicle category in ROPE3D, formula depth

predictions during inference slightly decreased detection accuracy. This decrease might be due to higher rotation and dimension estimation errors in the Big Vehicle class, which propagate into the formula depth predictions. To remedy this problem, the warmup strategy called Hierarchical Temporal Learning (HTL), proposed in GUPNet [16], can be investigated. This strategy could ensure that the network uses formula depth predictions only after generating reliable predictions for 3D properties such as dimensions and rotation.

After all depth estimations are obtained, we suggested robustly combining them using uncertainty, as proposed in MonoDDE [18]. From the analysis in Section 5.4, we can clearly see that the suggested combination approach works well to detect and remove outliers, which means the learned uncertainties are useful. The ablation study in Section 6.5.2 indicates combining direct depth, keypoint heatmap depth, and formula depth improves the final detection results for both KITTI and ROPE3D. This suggests that our proposed method holds promise for generalized depth prediction from different views. However, these improvements are not considerably significant, indicating room for further enhancement.

The main reason for the lack of significant improvements might be how we combined the dense depth estimations. We separated the model learning part from the depth combination part. This means the model learns how to predict the accurate depth but does not learn to combine it. This separation may cause valuable depth predictions for the objects to be missing. Implementing an end-to-end learning network that has the ability to learn to combine depths could help remedy this.

This end-to-end learning can be achieved by introducing an attention mechanism. Using a similar architecture to the one used for attention in MonoXiver [20], internal depth predictions can be merged with object features and fed into the attention mechanism. With the help of the attention mechanism, the network can learn to select the most valuable depth estimations from direct, formula, and keypoint heatmap depth predictions and combine them effectively. By incorporating this architecture, the model can learn to predict more accurate depths in an end-to-end manner.

According to our findings, the proposed methods improve detection accuracy in various datasets and scenarios, addressing major problems with depth estimation. These findings point to several interesting areas for further research. Firstly, further refinement of the keypoint depth prediction approach could improve its effectiveness across different views and environments. Secondly, investigating end-to-end learning architectures, particularly those that include attention mechanisms, may improve the integration and utilization of various depth predictions. Furthermore, optimizing the perspective-n-point method for formula depth estimation by investigating warmup strategies like HTL could provide more accurate and robust depth predictions. By continuing to address the complexities of monocular depth estimation, future research can build on our findings to develop more robust and accurate systems for autonomous driving applications.

8 Conclusion

This thesis addressed the significant challenges of monocular depth prediction in autonomous driving, particularly in the presence of variable camera setups and viewing conditions. We aimed to improve depth estimation across diverse views by developing and evaluating three methods: virtual depth, full rotation learning, and keypoint information exploitation.

The virtual depth method efficiently mitigated depth-scale ambiguity by including focal length in the optimization process. This method provided a unified optimization target, resulting in significant improvements in detection accuracy across various camera configurations, which is especially important for infrastructure-based object detection systems. The allocentric full rotation method improved perspective-invariant learning by using an allocentric 3×3 rotation matrix, addressing the limitations of traditional methods that only take into account the yaw angle. Our experiments on the KITTI and ROPE3D datasets confirmed that these methods significantly improved detection performance for vehicle and infrastructure-side views, demonstrating their effectiveness in a variety of driving scenarios.

The keypoint exploitation approach produced a variety of results. By increasing the number of keypoints and investigating keypoint-to-center offset learning, we expected to improve geometric understanding and depth prediction accuracy. However, the additional keypoints add complexity and noise to the system, and Kpt2Center offset learning did not produce the desired results. Despite these challenges, keypoint depth heatmaps demonstrated potential, particularly for larger vehicle classes in ROPE3D, and while they improved results for cars, the improvements were not as significant.

The formula depth method, which incorporates predicted dimensions, full rotation, and a model center to estimate depth from keypoints, also demonstrated its utility in improving depth predictions. Furthermore, learning uncertainty values for the formula depth estimations helped robustly filter outliers, enhancing the reliability of the final depth predictions. Analysis reveals that the formula depth uncertainties behaved as expected in the majority of cases. While using formula depth predictions during inference improved detection accuracy for the Car class in both KITTI and ROPE3D datasets, the improvements were not significant. For larger vehicles in the ROPE3D dataset, the method showed a slight decrease in detection accuracy, likely due to higher rotation and dimension estimation errors. Despite these mixed results, the formula depth method's ability to provide additional depth clues remains a valuable asset, suggesting that further optimization and integration strategies, such as the HTL warmup strategy, could enhance its performance in future research.

Combining depth predictions from the different depth estimation methods, including direct depth, keypoint heatmap depth, and formula depth, using an uncertainty-based MonoDDE approach proved useful. This robust gathering approach helped to detect and remove outlier depth predictions, resulting in improved final detection results for both KITTI and ROPE3D

datasets. However, the improvements were not as significant as anticipated, which shows us the need for further refinement in the depth combination process. An end-to-end dense depth prediction and combination network with an attention mechanism could potentially address this issue by learning to combine dense depth estimations more effectively.

Our findings show that virtual depth and full rotation learning methods are reliable and useful under a variety of viewing conditions. On the other hand, while keypoint exploitation shows promise, it requires further refinement to fully realize its potential. The insights from this study suggest several areas for future research.

Firstly, further exploration of keypoint-based methods could improve the effectiveness of depth predictions across different views and environments. This might involve developing a better association algorithm for a keypoint heatmap-based approach. Secondly, implementing end-to-end learning networks, particularly those incorporating attention mechanisms, could enhance the integration and utilization of densely predicted depths. This approach would allow the network to learn not only accurate depth predictions from center and keypoints but also the optimal way to combine them. Thirdly, investigating warmup strategies such as HTL may improve the PnP method for keypoint depth estimation, resulting in more accurate and robust depth predictions. This would be especially useful in complex urban environments with diverse object densities and geometries. Finally, expanding the evaluation of proposed methods to include datasets other than KITTI and ROPE3D could provide a better understanding of their generalizability and robustness in real-world scenarios.

In conclusion, this research has shown that innovative approaches to overcoming the challenges of monocular depth prediction, including the use of keypoints, can enhance the effectiveness of 3D object detection systems in autonomous driving across different views. This study's findings and proposed future directions offer a promising framework for developing safer and more reliable autonomous vehicles.

List of Figures

3.1	The Illustration of Used Base Detector MonoCon [19]	7
4.1	The illustration of virtual camera properties [5]	11
4.2	Allocentric and Egocentric Representations Under Constant Rotations [5] . . .	12
4.3	All Keypoints After Addition: Default Keypoints (Blue), Edge Center Keypoints (Green), Area Center Keypoints (Red)	13
4.4	The Visualisations of Center-to-Keypoint and Keypoint-to-Center Concepts . .	14
4.5	Extracting Keypoint Depth Predictions at Inference	16
4.6	Loss of Object Association During Keypoint Detection	17
4.7	Dynamic Search Radius by Keypoint Depth	18
4.8	Comparison of Static (Left) and Dynamic(Right) Seach Radius	18
4.9	Learning Formula Depth Prediction Uncertainty	21
5.1	Comparison of Keypoint Depth Heatmap Prediction Distribution for Separate Channel-Shared Channel Implementation	24
5.2	Locations of Nine Key Points Displayed on the Image	25
5.3	The Percentages of Keypoint Estimations That is Better Than Direct	26
5.4	Comparison of Keypoint Heatmap Depth and Direct Depth Error	27
5.5	Formula Uncertainty Prediction Analysis	28
5.6	Comparison of MonoDDE Combined Formula Depth and Direct Depth Error	29
5.7	Comparative Analysis of Depth Estimation Error: Direct vs. Dense MonoDDE Method	30

List of Tables

5.1	Comparison of Evaluation Results with and without Keypoint Allocentric and Dimension Heads on KITTI	23
5.2	Keypoint Statistics Including Use Count, Prediction Range, Mean Error, and Mean Sigma	25
5.3	Depth Prediction Statistics by Different Methods	28
5.4	Depth Prediction Error Statistics for Direct and Dense MonoDDE Ensemble Methods Across Ranges	30
6.1	Auxiliary Keypoints Evaluation Results for AP40 IOU 0.7 CARS.	33
6.2	Keypoint2Center Offset Learning Evaluation Results for AP40 IOU 0.7 CARS	34
6.3	Comparison of 2D Euclidean Distances Between Ground Truth and Various Keypoint Estimation Methods	34
6.4	KITTI Ablation Study Results for Virtual Depth and Allocentric Rotation on Car Category	35
6.5	Rope3D Virtual Depth and Allocentric Rotation Ablation Study for Car	35
6.6	Rope3D Virtual Depth and Allocentric Rotation Ablation Study for Big Vehicle	35
6.7	KITTI Car Category Ablation Study for Dense Depth Estimation	36
6.8	ROPE3D Car Category Ablation Study for Dense Depth Estimation	36
6.9	ROPE3D Big Vehicle Category Ablation Study for Dense Depth Estimation	37

Acronyms

AAS	Average Area Similarity. 32
ACS	Average Ground Center Similarity. 32
AGS	Average Ground Points Similarity. 32
AOS	Average Orientation Similarity. 32, 35
AP	Average Precision. 31–33
BEV	Bird’s-Eye-View. 4
CAD	Computer-Aided Design. 3
Center2Kpt	Center to Keypoint. 14, 20, 21, 34, 38, 39
GT	Ground Truth. 21, 24, 26–28, 30, 32, 34
HTL	Hierarchical Temporal Learning. 40–42
IoU	Intersection over Union. 16, 24, 31, 32
Kpt2Center	Keypoint to Center. 14, 33, 38, 39, 41
LiDAR	Light Detection and Ranging. 1, 3, 4
LSS	Learnable Sample Selection. 4
MAE	Mean Absolute Error. 26, 29, 30
MLP	Multi Layer Perceptron. 4
PnP	Perspective-n-Point. 1, 7, 12, 19, 42
V2X	Vehicle-to-Everything. 5

Bibliography

- [1] G. P. Meyer, A. Laddha, E. Kee, C. Vallespi-Gonzalez, and C. K. Wellington. “Lasernet: An efficient probabilistic 3d object detector for autonomous driving”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2019, pp. 12677–12686.
- [2] C. R. Qi, O. Litany, K. He, and L. J. Guibas. “Deep hough voting for 3d object detection in point clouds”. In: *proceedings of the IEEE/CVF International Conference on Computer Vision*. 2019, pp. 9277–9286.
- [3] S. Shi, Z. Wang, J. Shi, X. Wang, and H. Li. “From points to parts: 3d object detection from point cloud with part-aware and part-aggregation network”. In: *IEEE transactions on pattern analysis and machine intelligence* 43.8 (2020), pp. 2647–2664.
- [4] S. Shi, X. Wang, and H. Li. “Pointtrcnn: 3d object proposal generation and detection from point cloud”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2019, pp. 770–779.
- [5] G. Brazil, A. Kumar, J. Straub, N. Ravi, J. Johnson, and G. Gkioxari. “Omni3d: A large benchmark and model for 3d object detection in the wild”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2023, pp. 13154–13164.
- [6] Z. Liu, D. Zhou, F. Lu, J. Fang, and L. Zhang. “Autoshape: Real-time shape-aware monocular 3d object detection”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2021, pp. 15641–15650.
- [7] F. Chabot, M. Chaouch, J. Rabarisoa, C. Teuliere, and T. Chateau. “Deep manta: A coarse-to-fine many-task network for joint 2d and 3d vehicle analysis from monocular image”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017, pp. 2040–2049.
- [8] Y. Li, Y. Chen, J. He, and Z. Zhang. “Densely constrained depth estimator for monocular 3d object detection”. In: *European Conference on Computer Vision*. Springer. 2022, pp. 718–734.
- [9] X. Chen, K. Kundu, Z. Zhang, H. Ma, S. Fidler, and R. Urtasun. “Monocular 3d object detection for autonomous driving”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 2147–2156.
- [10] M. Ding, Y. Huo, H. Yi, Z. Wang, J. Shi, Z. Lu, and P. Luo. “Learning depth-guided convolutions for monocular 3d object detection”. In: *Proceedings of the IEEE/CVF Conference on computer vision and pattern recognition workshops*. 2020, pp. 1000–1001.
- [11] J. M. U. Vianney, S. Aich, and B. Liu. “Refinedmpl: Refined monocular pseudolidar for 3d object detection in autonomous driving”. In: *arXiv preprint arXiv:1911.09712* (2019).

- [12] Y. Wang, W.-L. Chao, D. Garg, B. Hariharan, M. Campbell, and K. Q. Weinberger. "Pseudo-lidar from visual depth estimation: Bridging the gap in 3d object detection for autonomous driving". In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2019, pp. 8445–8453.
- [13] X. Weng and K. Kitani. "Monocular 3d object detection with pseudo-lidar point cloud". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops*. 2019, pp. 0–0.
- [14] Z. Qin, J. Wang, and Y. Lu. "Monogrnet: A geometric reasoning network for monocular 3d object localization". In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 33. 01. 2019, pp. 8851–8858.
- [15] G. Brazil and X. Liu. "M3d-rpn: Monocular 3d region proposal network for object detection". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2019, pp. 9287–9296.
- [16] Y. Lu, X. Ma, L. Yang, T. Zhang, Y. Liu, Q. Chu, J. Yan, and W. Ouyang. "Geometry uncertainty projection network for monocular 3d object detection". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2021, pp. 3111–3121.
- [17] Y. Zhang, J. Lu, and J. Zhou. "Objects are different: Flexible monocular 3d object detection". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2021, pp. 3289–3298.
- [18] Z. Li, Z. Qu, Y. Zhou, J. Liu, H. Wang, and L. Jiang. "Diversity matters: Fully exploiting depth clues for reliable monocular 3d object detection". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022, pp. 2791–2800.
- [19] X. Liu, N. Xue, and T. Wu. "Learning auxiliary monocular contexts helps monocular 3d object detection". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 36. 2. 2022, pp. 1810–1818.
- [20] X. Liu, C. Zheng, K. B. Cheng, N. Xue, G.-J. Qi, and T. Wu. "Monocular 3d object detection with bounding box denoising in 3d by perceiver". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2023, pp. 6436–6446.
- [21] L. Yang, J. Yu, X. Zhang, J. Li, L. Wang, Y. Huang, C. Zhang, H. Wang, and Y. Li. "MonoGAE: Roadside monocular 3D object detection with ground-aware embeddings". In: *arXiv preprint arXiv:2310.00400* (2023).
- [22] H. Shi, C. Pang, J. Zhang, K. Yang, Y. Wu, H. Ni, Y. Lin, R. Stiefelhagen, and K. Wang. "Cobev: Elevating roadside 3d object detection with depth and height complementarity". In: *arXiv preprint arXiv:2310.02815* (2023).
- [23] S. Fan, Z. Wang, X. Huo, Y. Wang, and J. Liu. "Calibration-free bev representation for infrastructure perception". In: *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE. 2023, pp. 9008–9013.

- [24] J. Jinrang, Z. Li, and Y. Shi. “MonoUNI: A unified vehicle and infrastructure-side monocular 3d object detection network with sufficient depth clues”. In: *Advances in Neural Information Processing Systems* 36 (2024).
- [25] Z. Li, J. Jia, and Y. Shi. “MonoLSS: Learnable Sample Selection For Monocular 3D Detection”. In: *arXiv preprint arXiv:2312.14474* (2023).
- [26] A. Geiger, P. Lenz, and R. Urtasun. “Are we ready for Autonomous Driving? The KITTI Vision Benchmark Suite”. In: *Conference on Computer Vision and Pattern Recognition (CVPR)*. 2012.
- [27] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom. “nuscenes: A multimodal dataset for autonomous driving”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2020, pp. 11621–11631.
- [28] P. Sun, H. Kretzschmar, X. Dotiwalla, A. Chouard, V. Patnaik, P. Tsui, J. Guo, Y. Zhou, Y. Chai, B. Caine, et al. “Scalability in perception for autonomous driving: Waymo open dataset”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2020, pp. 2446–2454.
- [29] X. Huang, X. Cheng, Q. Geng, B. Cao, D. Zhou, P. Wang, Y. Lin, and R. Yang. “The apolloscape dataset for autonomous driving”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition workshops*. 2018, pp. 954–960.
- [30] F. Yu, H. Chen, X. Wang, W. Xian, Y. Chen, F. Liu, V. Madhavan, and T. Darrell. “Bdd100k: A diverse driving dataset for heterogeneous multitask learning”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2020, pp. 2636–2645.
- [31] X. Ye, M. Shu, H. Li, Y. Shi, Y. Li, G. Wang, X. Tan, and E. Ding. “Rope3D: The Roadside Perception Dataset for Autonomous Driving and Monocular 3D Object Detection Task”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2022, pp. 21341–21350.
- [32] H. Yu, Y. Luo, M. Shu, Y. Huo, Z. Yang, Y. Shi, Z. Guo, H. Li, X. Hu, J. Yuan, et al. “Dair-v2x: A large-scale dataset for vehicle-infrastructure cooperative 3d object detection”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022, pp. 21361–21370.
- [33] A. Karpathy, G. Toderici, S. Shetty, T. Leung, R. Sukthankar, and L. Fei-Fei. “Large-scale video classification with convolutional neural networks”. In: *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*. 2014, pp. 1725–1732.
- [34] X. Ma, Y. Zhang, D. Xu, D. Zhou, S. Yi, H. Li, and W. Ouyang. “Delving into localization errors for monocular 3d object detection”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2021, pp. 4721–4730.
- [35] Z. Liu, Z. Wu, and R. Tóth. “Smoke: Single-stage monocular 3d object detection via keypoint estimation”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*. 2020, pp. 996–997.

- [36] K. Duan, S. Bai, L. Xie, H. Qi, Q. Huang, and Q. Tian. “Centernet: Keypoint triplets for object detection”. In: *Proceedings of the IEEE/CVF international conference on computer vision*. 2019, pp. 6569–6578.
- [37] P. Li, H. Zhao, P. Liu, and F. Cao. “Rtm3d: Real-time monocular 3d detection from object keypoints for autonomous driving”. In: *European Conference on Computer Vision*. Springer. 2020, pp. 644–660.
- [38] F. Yu, D. Wang, E. Shelhamer, and T. Darrell. “Deep layer aggregation”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 2403–2412.
- [39] Y. Chen, L. Tai, K. Sun, and M. Li. “Monopair: Monocular 3d object detection using pairwise spatial relationships”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2020, pp. 12093–12102.
- [40] A. Simonelli, S. R. Buló, L. Porzi, M. López-Antequera, and P. Kotschieder. “Disentangling monocular 3d object detection”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2019, pp. 1991–1999.
- [41] A. Mousavian, D. Anguelov, J. Flynn, and J. Kosecka. “3D Bounding Box Estimation Using Deep Learning and Geometry”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. July 2017.
- [42] J. K. Murthy, G. S. Krishna, F. Chhaya, and K. M. Krishna. “Reconstructing vehicles from a single image: Shape priors for road scene understanding”. In: *2017 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE. 2017, pp. 724–731.
- [43] M. Zeeshan Zia, M. Stark, and K. Schindler. “Are cars just 3d boxes?-jointly estimating the 3d shape of multiple objects”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2014, pp. 3678–3685.
- [44] Y. Zhou, C. Barnes, J. Lu, J. Yang, and H. Li. “On the continuity of rotation representations in neural networks”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2019, pp. 5745–5753.
- [45] A. Kundu, Y. Li, and J. M. Rehg. “3d-rcnn: Instance-level 3d object reconstruction via render-and-compare”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 3559–3568.
- [46] T. Asparouhov and B. Muthén. “Weighted least squares estimation with missing data”. In: *Mplus technical appendix 2010.1-10* (2010), p. 5.
- [47] Y. Zhou and O. Tuzel. “Voxelnet: End-to-end learning for point cloud based 3d object detection”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 4490–4499.